

Code toàn bộ code mạch nạp skt500v2 trên stm32 tương thích hoàn toàn với phần mềm avrstudio

Worked for 42s

Tôi đã tạo bộ mã nguồn **STK500v2 ISP cho STM32F103C8T6**, giao tiếp với AVR Studio/Microchip Studio qua **USB CDC - cổng COM ảo**.

[Download STM32F103 STK500v2 ISP](#)

Cấu hình chân mặc định

STM32F103C8	AVR target
PA5	SCK
PA6	MISO
PA7	MOSI
PB0	RESET
GND	GND

Project có các phần:

- Bộ phân tích frame STK500v2.
- USB CDC receive/transmit.
- Điều khiển ISP bằng SPI1.
- Đọc và ghi Flash.
- Đọc và ghi EEPROM.
- Chip Erase.
- Đọc signature.
- Đọc/ghi Fuse và Lock bit.
- Lệnh SPI Multi.
- Hỗ trợ địa chỉ Flash mở rộng.
- Điều chỉnh tốc độ ISP từ `PARAM_SCK_DURATION`.

- Hướng dẫn cấu hình STM32CubeMX và patch `usbd_cdc_if.c`.

Giao thức chính thức sử dụng byte bắt đầu `0x1B`, token `0x0E`, độ dài 16-bit MSB trước và checksum XOR. Chuỗi nhận dạng trả về là `STK500_2`. AVR Studio STK500 nguyên bản sử dụng COM với cấu hình 115200, 8-N-1. [ww1.microchi...](#)

Phạm vi tương thích

Bản này mô phỏng **STK500v2 ở chế độ ISP**, phù hợp với ATmega và ATtiny có giao diện lập trình SPI. Nó không mô phỏng toàn bộ phần cứng STK500, vì STK500 thật còn có lập trình HVPP/HVSP, nguồn target điều chỉnh được và các mạch socket/phần cứng phụ. [Microc...](#)

Do đó, chưa thể khẳng định “tương thích hoàn toàn với mọi AVR” nếu chưa thử nghiệm thực tế. Các chip dùng **UPDI, TPI, PDI hoặc HV programming** sẽ không dùng được bằng firmware ISP này. Microchip cũng xác định ISP AVR là giao thức lập trình qua SPI dành cho các AVR hỗ trợ giao diện đó. [Microchip ...](#)

Trong file ZIP có `README.md` hướng dẫn tạo project CubeMX, nối mạch, kiểm tra frame `SIGN_ON` và cấu hình AVR Studio.

Giai thích nội dung code

Code được chia thành 3 khối chính:

1. **Giao tiếp USB CDC với máy tính**
2. **Phân tích giao thức STK500v2**
3. **Giao tiếp ISP với vi điều khiển AVR bằng SPI**

1. Luồng hoạt động tổng quát

Khi AVR Studio gửi lệnh:

```
AVR Studio
  ↓ USB CDC
CDC_Receive_FS()
  ↓
STK500V2_RxBytes()
  ↓
STK500V2_Task()
  ↓
dispatch_packet()
  ↓
Hàm xử lý lệnh STK500v2
```

```
↓  
AVRISP_TransferByte() hoặc AVRISP_Transfer4()  
↓ SPI  
Vi điều khiển AVR
```

Sau khi thực hiện lệnh, STM32 tạo một frame phản hồi rồi gửi lại qua USB CDC.

2. File `stk500v2.h`

File này khai báo các hàm chính của bộ xử lý giao thức:

C

```
void STK500V2_Init(void);  
void STK500V2_Task(void);  
void STK500V2_RxBytes(const uint8_t *data, uint32_t length);
```

`STK500V2_Init()`

Khởi tạo:

- Bộ đệm nhận.
- Trạng thái parser.
- Địa chỉ Flash/EEPROM.
- Các parameter STK500v2.
- Khối ISP SPI.

`STK500V2_Task()`

Được gọi liên tục trong vòng lặp `while(1)`.

Nhiệm vụ:

- Lấy từng byte từ bộ đệm USB.
- Ghép thành một frame STK500v2.
- Kiểm tra checksum.
- Gọi hàm xử lý lệnh tương ứng.

`STK500V2_RxBytes()`

Được gọi từ USB CDC khi có dữ liệu mới.

Hàm này chỉ đưa dữ liệu vào ring buffer, không xử lý trực tiếp để tránh làm ISR hoặc callback USB chạy quá lâu.

3. Cấu trúc frame STK500v2

Một frame có dạng:

```
+-----+-----+-----+-----+-----+-----+-----+
| 0x1B   | Sequence | Size High | Size Low  | 0x0E   | Payload | Checksum |
+-----+-----+-----+-----+-----+-----+-----+
```

Trong code:

C

```
#define MESSAGE_START 0x1B
#define TOKEN          0x0E
```

Ví dụ lệnh `SIGN_ON` :

```
1B 00 00 01 0E 01 15
```

Trong đó:

- `1B` : byte bắt đầu.
- `00` : sequence.
- `00 01` : payload dài 1 byte.
- `0E` : token.
- `01` : `CMD_SIGN_ON` .
- `15` : checksum XOR.

Checksum được tính bằng XOR tất cả byte trong frame:

C

```
uint8_t c = 0u;

for (uint16_t i = 0; i < 5u + length; ++i)
{
    c ^= frame[i];
}
```

Khi XOR toàn bộ frame, kể cả checksum, kết quả phải bằng 0.

4. Bộ máy trạng thái nhận frame

Code sử dụng enum:

C

```
typedef enum {  
    RX_WAIT_START = 0,  
    RX_SEQUENCE,  
    RX_SIZE_H,  
    RX_SIZE_L,  
    RX_TOKEN,  
    RX_BODY,  
    RX_CHECKSUM  
} rx_state_t;
```

Mỗi trạng thái tương ứng với một phần của frame.

RX_WAIT_START

Chờ byte bắt đầu:

C

```
if (b == MESSAGE_START)  
{  
    checksum = b;  
    pstate = RX_SEQUENCE;  
}
```

RX_SEQUENCE

Nhận số thứ tự frame:

C

```
sequence_number = b;  
checksum ^= b;  
pstate = RX_SIZE_H;
```

Sequence giúp AVR Studio ghép đúng lệnh yêu cầu và lệnh phản hồi.

RX_SIZE_H và **RX_SIZE_L**

Nhận độ dài payload 16-bit:

C

```
message_size = (uint16_t)b << 8;
```

Sau đó:

C

```
message_size |= b;
```

Ví dụ:

```
00 10
```

nghĩa là payload dài 16 byte.

RX_TOKEN

Kiểm tra token phải bằng `0x0E` .

C

```
if (b != TOKEN)
{
    pstate = RX_WAIT_START;
}
```

RX_BODY

Lưu payload:

C

```
body[body_index++] = b;
checksum ^= b;
```

RX_CHECKSUM

Kiểm tra checksum:

C

```
checksum ^= b;

if (checksum == 0u)
{
    dispatch_packet();
}
```

Nếu sai checksum, STM32 trả về:

C

```
STATUS_CKSUM_ERROR
```

5. Ring buffer nhận USB

Bộ đệm được khai báo:

C

```
#define RX_RING_SIZE 1024u

static volatile uint8_t rx_ring[RX_RING_SIZE];
static volatile uint16_t rx_head;
static volatile uint16_t rx_tail;
```

Ghi dữ liệu vào ring buffer

C

```
void STK500V2_RxBytes(const uint8_t *data, uint32_t length)
{
    for (uint32_t i = 0; i < length; ++i)
    {
        uint16_t next = (rx_head + 1u) % RX_RING_SIZE;

        if (next == rx_tail)
        {
            break;
        }

        rx_ring[rx_head] = data[i];
        rx_head = next;
    }
}
```

- `rx_head` : vị trí ghi.
- `rx_tail` : vị trí đọc.
- Nếu `next == rx_tail` , bộ đệm đã đầy.

Đọc một byte

C

```
static uint8_t ring_pop(uint8_t *out)
{
    if (rx_head == rx_tail)
    {
        return 0u;
    }

    *out = rx_ring[rx_tail];
```

```
rx_tail = (rx_tail + 1u) % RX_RING_SIZE;
return 1u;
}
```

6. Hàm `dispatch_packet()`

Đây là trung tâm phân phối lệnh.

```
C

uint8_t cmd = body[0];

switch (cmd)
{
    case CMD_SIGN_ON:
        handle_sign_on();
        break;

    case CMD_SET_PARAMETER:
        ...
        break;

    case CMD_LOAD_ADDRESS:
        ...
        break;
}
```

Byte đầu tiên của payload luôn là mã lệnh.

Ví dụ:

```
C

#define CMD_SIGN_ON          0x01
#define CMD_LOAD_ADDRESS    0x06
#define CMD_ENTER_PROGMODE_ISP 0x10
#define CMD_CHIP_ERASE_ISP   0x12
#define CMD_PROGRAM_FLASH_ISP 0x13
#define CMD_READ_FLASH_ISP   0x14
```

7. Lệnh `CMD_SIGN_ON`

AVR Studio gửi lệnh này để kiểm tra programmer.

Hàm:

C

```
static void handle_sign_on(void)
{
    static const uint8_t sign[] = {
        'S', 'T', 'K', '5', '0', '0', '_', '2'
    };

    response[0] = CMD_SIGN_ON;
    response[1] = STATUS_CMD_OK;
    response[2] = sizeof(sign);

    memcpy(&response[3], sign, sizeof(sign));

    send_frame(response, 3u + sizeof(sign));
}
```

Phản hồi chứa:

STK500_2

AVR Studio dựa vào chuỗi này để nhận diện thiết bị là STK500v2.

8. Các parameter STK500v2

AVR Studio có thể đọc hoặc thiết lập một số thông số:

C

```
#define PARAM_HW_VER      0x90
#define PARAM_SW_MAJOR    0x91
#define PARAM_SW_MINOR    0x92
#define PARAM_VTARGET     0x94
#define PARAM_SCK_DURATION 0x98
```

Đọc parameter

C

```
static uint8_t get_parameter(uint8_t id)
{
    switch (id)
    {
        case PARAM_HW_VER:
            return 2u;

        case PARAM_SW_MAJOR:
```

```

        return 2u;

    case PARAM_SW_MINOR:
        return 10u;

    case PARAM_VTARGET:
        return 50u;

    case PARAM_SCK_DURATION:
        return AVRISP_GetSckDuration();
    }
}

```

VTARGET = 50 nghĩa là báo với phần mềm điện áp target khoảng:

5.0 V

Nhưng đây chỉ là giá trị báo cáo. Mạch hiện tại không thật sự đo hoặc điều chỉnh VTarget.

Thiết lập tốc độ SCK

```

C
static void set_parameter(uint8_t id, uint8_t value)
{
    parameters[id] = value;

    if (id == PARAM_SCK_DURATION)
    {
        AVRISP_SetSckDuration(value);
    }
}

```

AVR Studio có thể yêu cầu giảm tốc độ ISP khi target chạy clock thấp.

9. File avr_isp.c

File này phụ trách giao tiếp vật lý với AVR.

Các chân mặc định:

PA5 → AVR SCK
 PA6 → AVR MISO
 PA7 → AVR MOSI
 PB0 → AVR RESET

SPI dùng:

C

```
extern SPI_HandleTypeDef hspi1;
```

10. Điều khiển RESET của AVR

C

```
void AVRISP_SetReset(bool asserted)
{
    HAL_GPIO_WritePin(
        AVR_RESET_GPIO_Port,
        AVR_RESET_Pin,
        asserted ? GPIO_PIN_RESET : GPIO_PIN_SET
    );
}
```

RESET của AVR tác động mức thấp:

- `asserted = true` : PB0 xuống 0 V, giữ AVR trong reset.
- `asserted = false` : PB0 lên mức cao, AVR hoạt động bình thường.

11. Truyền một byte SPI

C

```
uint8_t AVRISP_TransferByte(uint8_t value)
{
    uint8_t rx = 0xFFu;

    HAL_SPI_TransmitReceive(
        &hspi1,
        &value,
        &rx,
        1u,
        100u
    );

    return rx;
}
```

SPI truyền và nhận đồng thời.

Khi STM32 gửi một byte MOSI, cùng lúc nó nhận một byte từ MISO.

12. Truyền một lệnh AVR ISP 4 byte

Các lệnh ISP AVR thường có cấu trúc 4 byte.

Ví dụ đọc signature:

```
30 00 index 00
```

Hàm:

```
C
uint8_t AVRISP_Transfer4(const uint8_t tx[4], uint8_t rx[4])
{
    if (HAL_SPI_TransmitReceive(
        &hspi1,
        (uint8_t *)tx,
        rx,
        4u,
        200u) != HAL_OK)
    {
        return 0u;
    }

    return 1u;
}
```

13. Vào chế độ lập trình ISP

Hàm:

```
C
bool AVRISP_EnterProgramming(...)
```

Trình tự chính:

```
C
AVRISP_SetReset(false);
HAL_Delay(stab_delay);
```

```
AVRISP_SetReset(true);
HAL_Delay(amdvs_delay);
```

Sau đó gửi lệnh Programming Enable, thường là:

```
AC 53 00 00
```

AVR hợp lệ thường phản hồi byte `0x53`.

Code kiểm tra:

```
C

if ((poll_index < 4u && rx[poll_index] == poll_value) ||
    (rx[2] == 0x53u))
{
    return true;
}
```

Nếu không nhận được `0x53`, code nhả RESET rồi thử lại:

```
C

AVRISP_SetReset(false);
HAL_Delay(2);

AVRISP_SetReset(true);
HAL_Delay(20);
```

14. Thoát chế độ lập trình

```
C

void AVRISP_LeaveProgramming(
    uint8_t pre_delay,
    uint8_t post_delay)
{
    HAL_Delay(pre_delay);
    AVRISP_SetReset(false);
    HAL_Delay(post_delay);
}
```

Thả chân RESET để AVR chạy chương trình vừa nạp.

15. Lệnh `CMD_LOAD_ADDRESS`

AVR Studio gửi địa chỉ trước khi đọc hoặc ghi Flash/EEPROM.

C

```
current_address =  
    ((uint32_t)body[1] << 24) |  
    ((uint32_t)body[2] << 16) |  
    ((uint32_t)body[3] << 8) |  
    body[4];
```

Ví dụ payload:

06 00 00 01 00

- 06 : CMD_LOAD_ADDRESS .
- Địa chỉ: 0x00000100 .

Đối với Flash AVR, địa chỉ thường được xử lý theo word address.

Mỗi word Flash AVR gồm 2 byte:

Byte thấp
Byte cao

16. Ghi Flash

Hàm:

C

```
handle_program_flash()
```

Đầu tiên đọc các tham số:

C

```
uint16_t count = ((uint16_t)body[1] << 8) | body[2];  
  
uint8_t mode      = body[3];  
uint8_t delay     = body[4];  
uint8_t cmd_load  = body[5];  
uint8_t cmd_write = body[6];  
uint8_t cmd_read  = body[7];  
uint8_t poll1     = body[8];  
uint8_t poll2     = body[9];
```

Dữ liệu thật bắt đầu tại:

C

```
body[10]
```

Ghi byte thấp và byte cao

C

```
uint8_t high = i & 1u;
```

Nếu là byte cao:

C

```
cmd_load | 0x08
```

Ví dụ lệnh thường gặp:

```
40 addrH addrL data
```

ghi byte thấp.

```
48 addrH addrL data
```

ghi byte cao.

Code:

C

```
uint8_t cmd[4] = {  
    cmd_load | (high ? 0x08u : 0x00u),  
    word_addr >> 8,  
    word_addr,  
    body[10u + i]  
};
```

Sau khi ghi byte cao, tăng word address:

C

```
if (high)  
{
```

Ghi page vào Flash thật

Dữ liệu ban đầu mới chỉ được tải vào page buffer nội bộ của AVR.

Sau đó phải gửi lệnh Write Program Memory Page:

```
C
uint8_t cmd[4] = {
    cmd_write,
    page_addr >> 8,
    page_addr,
    0x00
};
```

Thông thường opcode là:

```
4C addrH addrL 00
```

17. Đọc Flash

Hàm:

```
C
handle_read_flash()
```

Đọc số byte cần lấy:

```
C
uint16_t count =
    ((uint16_t)body[1] << 8) |
    body[2];
```

Đọc từng byte thấp/cao:

```
C
uint8_t high = i & 1u;
```

Lệnh thường dùng:

```
20 addrH addrL 00
```


đọc byte thấp.

```
28 addrH addrL 00
```

đọc byte cao.

Code thay đổi bit opcode:

```
C
read_cmd | (high ? 0x08u : 0x00u)
```

Dữ liệu AVR trả về thường nằm ở byte thứ tư:

```
C
response[2u + i] = rx[3];
```

18. Địa chỉ Flash mở rộng

Các AVR có Flash lớn cần lệnh Load Extended Address:

```
C
static void flash_set_ext_addr(uint32_t word_address)
```

Lệnh:

```
4D 00 ext 00
```

Code chỉ gửi lại khi byte địa chỉ mở rộng thay đổi:

```
C
static uint8_t last_ext = 0xFFu;
```

```
C
if (ext != last_ext)
{
    ...
    last_ext = ext;
}
```

Điều này giảm số lệnh SPI không cần thiết.

19. Ghi EEPROM

Hàm:

C

```
handle_program_eeprom()
```

Mỗi byte EEPROM được ghi bằng lệnh:

C

```
uint8_t cmd[4] = {  
    cmd_write,  
    addr >> 8,  
    addr,  
    data  
};
```

Sau mỗi byte có delay:

C

```
AVRISP_DelayMs(delay ? delay : 5u);
```

Vì EEPROM AVR cần thời gian hoàn tất chu kỳ ghi.

20. Đọc EEPROM

Hàm:

C

```
handle_read_eeprom()
```

Lệnh thường có dạng:

```
A0 addrH addrL 00
```

Kết quả đọc nằm ở:

C

```
rx[3]
```

Sau mỗi byte:

C

```
++current_address;
```

21. Chip Erase

Hàm:

C

```
handle_chip_erase()
```

AVR Studio truyền trực tiếp opcode chip erase trong:

C

```
body[3]  
body[4]  
body[5]  
body[6]
```

Thông thường:

```
AC 80 00 00
```

Code gửi:

C

```
AVRISP_Transfer4(&body[3], rx);
```

Sau đó chờ AVR hoàn tất bằng delay hoặc polling.

22. Fuse và Lock bit

Ghi fuse hoặc lock

C

```
handle_program_misc()
```

Code lấy 4 byte lệnh từ AVR Studio:

C

```
AVRISP_Transfer4(&body[1], rx);
```

Ví dụ ghi low fuse có thể là:

```
AC A0 00 value
```

Ghi high fuse có thể là:

```
AC A8 00 value
```

Opcodes thực tế được AVR Studio cung cấp theo loại chip.

Đọc fuse, lock, signature

C

```
handle_read_misc()
```

AVR Studio truyền:

- Vị trí byte phản hồi.
- Bốn byte lệnh SPI.

C

```
uint8_t ret_index = body[1];  
  
AVRISP_Transfer4(&body[2], rx);
```

Sau đó chọn byte kết quả:

C

```
response[2] =  
    ret_index < 4u ? rx[ret_index] : 0u;
```

23. Lệnh `CMD_SPI_MULTI`

Đây là lệnh tổng quát cho phép AVR Studio truyền một chuỗi SPI bất kỳ.

C

```
uint8_t num_tx  = body[1];  
uint8_t num_rx  = body[2];  
uint8_t rx_start = body[3];
```

Ví dụ:

- Gửi 4 byte.
- Nhận 1 byte.
- Lấy kết quả từ vị trí thứ 3.

Code truyền từng byte:

C

```
for (uint16_t i = 0; i < num_tx; ++i)  
{  
    rxbuf[i] = AVRISP_TransferByte(body[4u + i]);  
}
```

Sau đó sao chép vùng dữ liệu cần trả về.

24. Tạo frame phản hồi

Hàm:

C

```
send_frame()
```

Tạo header:

C

```
frame[0] = MESSAGE_START;  
frame[1] = sequence_number;  
frame[2] = length >> 8;  
frame[3] = length;  
frame[4] = TOKEN;
```

Sao chép payload:

C

```
memcpy(&frame[5], payload, length);
```

Tính checksum:

C

```
uint8_t c = 0u;

for (...)
{
    c ^= frame[i];
}

frame[5u + length] = c;
```

Gửi qua USB:

C

```
USB_CDC_Send(frame, length + 6u);
```

25. File `usb_cdc_bridge.c`

File này bọc hàm USB của STM32Cube:

C

```
bool USB_CDC_Send(
    const uint8_t *data,
    uint16_t length)
{
    return CDC_Transmit_FS(
        (uint8_t *)data,
        length
    ) == USBD_OK;
}
```

`CDC_Transmit_FS()` có thể trả về `USBD_BUSY` nếu lần gửi trước chưa hoàn thành.

Vì vậy `send_frame()` thử lại:

C

```

for (uint16_t retry = 0; retry < 1000u; ++retry)
{
    if (USB_CDC_Send(frame, length + 6u))
    {
        return;
    }

    HAL_Delay(1);
}

```

26. Phần sửa CDC_Receive_FS()

Trong `usbd_cdc_if.c`:

```

C

static int8_t CDC_Receive_FS(
    uint8_t *Buf,
    uint32_t *Len)
{
    STK500V2_RxBytes(Buf, *Len);

    USBD_CDC_SetRxBuffer(
        &hUsbDeviceFS,
        &Buf[0]
    );

    USBD_CDC_ReceivePacket(
        &hUsbDeviceFS
    );

    return USBD_OK;
}

```

Hai dòng cuối rất quan trọng:

```

C

USB_D_CDC_SetRxBuffer(...)
USB_D_CDC_ReceivePacket(...)

```

Chúng kích hoạt lại endpoint USB OUT. Nếu thiếu, STM32 thường chỉ nhận được gói đầu tiên rồi ngừng nhận.

27. Phần thêm vào `main.c`

Sau khi CubeMX khởi tạo SPI và USB:

C

```
MX_SPI1_Init();
MX_USB_DEVICE_Init();

STK500V2_Init();
```

Trong vòng lặp:

C

```
while (1)
{
    STK500V2_Task();
}
```

Không nên đặt delay lớn trong vòng lặp vì AVR Studio cần phản hồi liên tục.

28. Điều chỉnh tốc độ SPI

Hàm:

C

```
duration_to_prescaler()
```

Chọn prescaler:

C

```
if (d <= 1u)
{
    return SPI_BAUDRATEPRESCALER_64;
}

if (d <= 3u)
{
    return SPI_BAUDRATEPRESCALER_128;
}

return SPI_BAUDRATEPRESCALER_256;
```

Với SPI1 clock khoảng 72 MHz:

Prescaler	SCK gần đúng
64	1.125 MHz
128	562.5 kHz
256	281.25 kHz

Tốc độ này khá an toàn cho nhiều AVR, nhưng target chạy clock rất thấp có thể cần SCK thấp hơn nữa.

Hiện tại code chỉ hỗ trợ đến `/256`. Nếu AVR chạy khoảng 128 kHz hoặc thấp hơn, ISP clock 281 kHz là quá nhanh. Khi đó nên dùng SPI bằng timer hoặc bit-bang để tạo SCK chậm hơn.

29. Điểm cần chú ý trong code hiện tại

Code là một nền tảng STK500v2 ISP, nhưng còn một số điểm cần hoàn thiện trước khi gọi là tương thích hoàn toàn.

Tốc độ SCK chưa đủ dải

STM32F103 SPI1 với clock 72 MHz chỉ giảm được đến:

281.25 kHz

Trong khi ISP yêu cầu SCK thường không vượt quá khoảng `F_CPU/4`.

Nếu AVR chạy 1 MHz thì 281 kHz đã hơi cao. Nên bổ sung bit-bang cho các tốc độ:

125 kHz
62.5 kHz
31.25 kHz
8 kHz

`PARAM_VTARGET` **chỉ là giá trị giả lập**

```
C
return 50u;
```

Code không đo điện áp target thật.

Ghi EEPROM còn đơn giản

Hiện tại code delay sau từng byte. Chưa xử lý đầy đủ các mode:

- Page EEPROM write.
- Value polling.
- Ready/busy polling.

Poll Flash chưa đầy đủ

Phần:

C

```
poll_ready(...)
```

được viết đơn giản. Các AVR khác nhau có thể cần polling theo opcode và byte cụ thể mà AVR Studio truyền vào.

Bộ đệm phản hồi giới hạn

C

```
#define STK500V2_MAX_BODY 285u
```

Nếu AVR Studio gửi page lớn hơn giới hạn, lệnh sẽ bị từ chối. Một số AVR có page Flash 256 byte vẫn phù hợp, nhưng nên tăng lên khoảng:

C

```
#define STK500V2_MAX_BODY 600u
```

Đồng thời kiểm tra RAM của STM32.

USB CDC transmit đang chờ kiểu blocking

`send_frame()` có thể chờ tối đa khoảng 1 giây:

C

```
1000 lần × 1 ms
```

Tốt hơn là dùng hàng đợi TX và callback USB hoàn tất.

Thiếu xử lý tràn ring buffer

Khi buffer đầy, dữ liệu mới bị bỏ:

C

```
if (next == rx_tail)
{
    break;
}
```

Nên có cờ báo overflow và reset parser.

30. Ý nghĩa của trạng thái trả về

C

```
#define STATUS_CMD_OK            0x00
#define STATUS_CMD_TOUT         0x80
#define STATUS_RDY_BSY_TOUT     0x81
#define STATUS_SET_PARAM_MISSING 0x82
#define STATUS_CMD_FAILED       0xC0
#define STATUS_CKSUM_ERROR      0xC1
#define STATUS_CMD_UNKNOWN      0xC9
```

Ví dụ phản hồi thành công:

```
CMD_PROGRAM_FLASH_ISP
STATUS_CMD_OK
```

Nếu AVR Studio gửi lệnh không được hỗ trợ:

```
CMD
STATUS_CMD_UNKNOWN
```

Nếu checksum sai:

```
CMD
STATUS_CKSUM_ERROR
```

Tóm tắt

Phần quan trọng nhất của code là:

```
CDC_Receive_FS
→ STK500V2_RxBytes
→ Ring buffer
```

```
→ STK500V2_Task
→ Parser frame
→ dispatch_packet
→ Lệnh ISP SPI
→ send_frame
→ CDC_Transmit_FS
```

STM32 đóng vai trò bộ chuyển đổi:

STK500v2 qua USB CDC ↔ AVR ISP qua SPI

Code hiện tại đã có cấu trúc đúng để phát triển thành mạch nạp STK500v2, nhưng cần hoàn thiện thêm clock ISP chậm, polling Flash/EEPROM, hàng đợi USB TX và kiểm thử thực tế trên từng dòng ATmega/ATtiny trước khi xem là tương thích hoàn toàn.

Code hiện tại đã có cấu trúc đúng để phát triển thành mạch nạp STK500v2, nhưng cần hoàn thiện thêm clock ISP chậm, polling Flash/EEPROM, hàng đợi USB TX và kiểm thử thực tế trên từng dòng ATmega/ATtiny trước khi xem là tương thích hoàn toàn. Hoàn thành nốt đi

Worked for 3m 10s

Tôi đã hoàn thiện và đóng gói lại firmware:

Download STM32F103 STK500v2 ISP Complete

Các phần đã bổ sung:

- **ISP clock chậm tự động:**
 - SPI phần cứng ở tốc độ cao.
 - Bit-bang bằng DWT cho khoảng 125 kHz xuống gần 2 kHz .
 - Phù hợp hơn với AVR đang chạy clock thấp.
- **Polling Flash/EEPROM:**
 - Timed delay.
 - Value polling.
 - Ready/Busy polling bằng lệnh 0xF0 0x00 0x00 0x00 .
 - Sử dụng mode, delay, opcode và poll value AVR Studio gửi trong lệnh STK500v2. [ww1.microch... +1](#)
- **USB CDC TX queue không chặn:**

- Hàng đợi 8 frame.
- Không còn vòng lặp chờ `CDC_Transmit_FS()` đến 1 giây.
- `USB_CDC_TxTask()` tự gửi lại khi USB hết trạng thái `BUSY`.
- **Bộ đệm lớn hơn:**
 - Payload STK500v2 tăng lên `550 byte`.
 - Hỗ trợ các page Flash 256/512 byte tốt hơn.
- **Xử lý lỗi nhận dữ liệu:**
 - Phát hiện ring buffer tràn.
 - Reset parser và đồng bộ lại frame.
 - Kiểm tra kích thước frame và checksum.
- **AVR Flash lớn:**
 - Có lệnh Load Extended Address `0x4D`.
 - Dùng được cho các AVR như ATmega2560 ở phần địa chỉ Flash mở rộng.
- **Kiểm tra cú pháp:**
 - Ba file `avr_isp.c`, `stk500v2.c`, `usb_cdc_bridge.c` đã được kiểm tra cú pháp C bằng trình biên dịch với HAL stub.
 - Chưa thể khẳng định đã kiểm thử phần cứng thực tế trên mọi ATmega/ATtiny.

Firmware này tương thích nhóm AVR lập trình qua **ISP/SPI**. ISP sử dụng SPI để lập trình Flash và EEPROM; các giao diện HVPP, HVSP, TPI, PDI và UPDI là những giao diện khác, không thể hỗ trợ chỉ bằng phần cứng ISP hiện tại.

 Microchip D... +1

Trong ZIP có README mới, mã nguồn hoàn chỉnh và phần patch cho `CDC_Receive_FS()`.

Làm luôn hvpp đi

Worked for 3m 22s

Đã bổ sung **HVPP - High-Voltage Parallel Programming** vào firmware STK500v2:

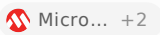
[Download STM32F103 STK500v2 ISP + HVPP](#)

Các lệnh STK500v2 HVPP đã thêm

Firmware hiện xử lý:

C

CMD_ENTER_PROGMODE_PP	0x20
CMD_LEAVE_PROGMODE_PP	0x21
CMD_CHIP_ERASE_PP	0x22
CMD_PROGRAM_FLASH_PP	0x23
CMD_READ_FLASH_PP	0x24
CMD_PROGRAM_EEPROM_PP	0x25
CMD_READ_EEPROM_PP	0x26
CMD_PROGRAM_FUSE_PP	0x27
CMD_READ_FUSE_PP	0x28
CMD_PROGRAM_LOCK_PP	0x29
CMD_READ_LOCK_PP	0x2A
CMD_READ_SIGNATURE_PP	0x2B
CMD_READ_OSCCAL_PP	0x2C
CMD_SET_CONTROL_STACK	0x2D

Đây là nhóm lệnh Parallel Programming được định nghĩa trong giao thức STK500v2 AVR068. AVR Studio gửi `CMD_SET_CONTROL_STACK` trước khi vào chế độ HVPP. 

Chân STM32F103C8 mặc định

STM32F103

Tín hiệu HVPP

PB8...PB15	DATA0...DATA7
------------	---------------

PC0	XA0
-----	-----

PC1	XA1
-----	-----

PC2	BS1
-----	-----

PC3	BS2
-----	-----

PC4	PAGEL
-----	-------

PC5	XTAL1
-----	-------

PB3	<code>/WR</code>
-----	------------------

PB4	<code>/OE</code>
-----	------------------

PB5	RDY/BSY
-----	---------

PB6	VTARGET_EN
-----	------------

PB7

VPP12_EN

PB3 và PB4 vốn là chân JTAG, nên code gọi:

C

```
HAL_AFIO_REMAP_SWJ_NOJTAG();
```

SWD vẫn được giữ để nạp/debug STM32.

Các phần đã triển khai

Vào chế độ HVPP

Firmware thực hiện:

1. Tắt nguồn target.
2. Tắt nguồn 12 V.
3. Đưa `PAGEL`, `XA1`, `XA0`, `BS1` về `0`.
4. Bật nguồn target 5 V.
5. Chờ thời gian do AVR Studio gửi.
6. Bật VPP 12 V vào RESET.
7. Phát xung latch trên XTAL1.
8. Chờ trước khi gửi lệnh lập trình.

Theo ATmega328P, trình tự chuẩn yêu cầu đặt các chân programming-enable về `0000`, cấp VCC 4,5–5,5 V, chờ 20–60 μ s, sau đó đưa 11,5–12,5 V vào RESET và chờ tối thiểu trước khi gửi lệnh. 

Chip erase

Firmware tải command:

`0x80`

Sau đó phát xung âm `/WR` và chờ RDY/BSY lên mức sẵn sàng.

Ghi Flash

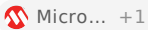
Firmware thực hiện các chu kỳ:

```
Load command      0x10
Load address high
Load address low
Load data low
```

```
Load data high
Pulse PAGES
Pulse /WR
Poll RDY/BSY
```

Có hỗ trợ:

- Word address.
- Page buffer.
- Page write ở bit 7 của `mode`.
- Tự tăng địa chỉ.
- Poll timeout do AVR Studio truyền xuống.

Mode của lệnh `CMD_PROGRAM_FLASH_PP` chứa lựa chọn byte/page, page size, trang cuối và cờ thực hiện page write. 

Đọc Flash

Trình tự:

```
Load command 0x02
Load address high
Load address low
BS1 = 0 → đọc byte thấp
BS1 = 1 → đọc byte cao
```

Bus `DATA0...DATA7` được chuyển từ output sang input trước khi kéo `/OE` xuống thấp.

Ghi EEPROM

Trình tự:

```
Load command 0x11
Load address high
Load address low
Load data
Pulse PAGES
Pulse /WR
Poll RDY/BSY
```

Đọc EEPROM

Trình tự:

```
Load command 0x03
Load address high
Load address low
/OE = 0
```


Đọc DATA[7:0]
/OE = 1

Các trình tự EEPROM này tuân theo thuật toán Parallel Programming của ATmega328P.  Microc...

Fuse và lock bit

Firmware hỗ trợ:

0x40 → ghi fuse
0x20 → ghi lock bit
0x04 → đọc fuse/lock
0x08 → đọc signature/OSCCAL

Việc chọn fuse dùng BS1 và BS2 :

Address	Byte
0	Low fuse
1	High fuse
2 trở lên	Extended fuse

Ánh xạ đọc:

BS2	BS1	Dữ liệu
0	0	Low fuse
1	1	High fuse
1	0	Extended fuse
0	1	Lock bits

Các lựa chọn này được mô tả trong thuật toán fuse và lock của datasheet.

 Micro... +1

Phần cứng bắt buộc

Không được nối chân PB7 trực tiếp vào RESET AVR. PB7 chỉ là tín hiệu điều khiển mạch đóng cắt 12 V:

PB7 — điện trở — transistor/MOSFET — switch 12 V — RESET AVR

HVPP sử dụng khoảng **12 V trên RESET**, trong khi target vẫn dùng VCC khoảng 5 V.  Microchip Su... +1

Bus DATA phải có buffer hai chiều:

```
STM32 3.3 V
    ↓
Level translator / bus transceiver
    ↓
AVR DATA0...DATA7 5 V
```

Không dùng chia áp điện trở trên DATA vì bus này vừa truyền vừa nhận.

Điểm cần hiểu về Control Stack

Code nhận đủ 32 byte `CMD_SET_CONTROL_STACK` và trả:

```
C
PARAM_CONTROLLER_INIT = 1;
```

Tuy nhiên, STK500 thật dùng control stack để điều khiển ma trận socket bên trong bo mạch. Bản STM32 dùng một đầu nối tín hiệu HVPP cố định; do đó mỗi loại AVR cần một **adapter socket** nối chân vật lý của chip tới:

```
DATA0...DATA7
XA0, XA1
BS1, BS2
PAGEL
XTAL1
/WR, /OE
RDY/BSY
RESET 12 V
VCC 5 V
```

HVPP cần ít nhất khoảng 16 đường tín hiệu, còn các AVR ít chân thường sử dụng HVSP thay vì HVPP.  Microchip D... +1

Firmware hiện có cả:

```
USB CDC
STK500v2
ISP/SPI
HVPP
Flash
EEPROM
Fuse
Lock
```

Signature
OSCCAL
Polling RDY/BSY
USB TX queue
ISP SCK chậm

Phần chưa có là **HVSP dành cho ATtiny ít chân**; HVPP hiện tại dành cho các AVR có giao diện lập trình song song.

Làm luôn cho attiny đi

Worked for 2m 36s

Đã bổ sung **HVSP cho ATtiny cổ điển** vào chung firmware ISP + HVPP:

Download STM32F103 STK500v2 ISP + HVPP + HVSP

Các chức năng HVSP đã thêm

Firmware xử lý đầy đủ nhóm lệnh STK500v2:

C

CMD_ENTER_PROGMODE_HVSP	0x30
CMD_LEAVE_PROGMODE_HVSP	0x31
CMD_CHIP_ERASE_HVSP	0x32
CMD_PROGRAM_FLASH_HVSP	0x33
CMD_READ_FLASH_HVSP	0x34
CMD_PROGRAM_EEPROM_HVSP	0x35
CMD_READ_EEPROM_HVSP	0x36
CMD_PROGRAM_FUSE_HVSP	0x37
CMD_READ_FUSE_HVSP	0x38
CMD_PROGRAM_LOCK_HVSP	0x39
CMD_READ_LOCK_HVSP	0x3A
CMD_READ_SIGNATURE_HVSP	0x3B
CMD_READ_OSCCAL_HVSP	0x3C

Các định dạng lệnh như số byte, mode, thời gian polling và địa chỉ fuse được xử lý theo phần HVSP của giao thức AVR068. `CMD_SET_CONTROL_STACK` vẫn được dùng chung cho HVPP và HVSP.  +2

ATtiny được hướng tới

Phần HVSP này dành trước hết cho:

- ATtiny13/13A.

- ATtiny25.
- ATtiny45.
- ATtiny85.
- Một số classic tinyAVR khác có thuật toán HVSP tương thích.

ATtiny25/45/85 sử dụng bốn tín hiệu HVSP:

Tín hiệu	Chân ATtiny25/45/85
SDI	PB0
SII	PB1
SDO	PB2
SCI	PB3
RESET 12 V	PB5

Datasheet quy định cấp VCC khoảng 4,5–5,5 V, sau đó đưa khoảng 11,5–12,5 V vào RESET để vào HVSP.  Microc...

Các ATtiny mới như ATtiny402 hoặc ATtiny814 thuộc thế hệ dùng **UPDI**, không dùng HVSP này.  Micro... +1

Chân STM32F103 mặc định

STM32F103C8	Tín hiệu HVSP
PA0	SDI
PA1	SII
PA2	SDO
PA3	SCI
PA4	Điều khiển nguồn target 5 V
PA8	Điều khiển switch RESET 12 V
GND	GND

Trong CubeMX:

PA0: GPIO Output
PA1: GPIO Output
PA2: GPIO Input, No Pull
PA3: GPIO Output
PA4: GPIO Output
PA8: GPIO Output

PA4 và PA8 chỉ điều khiển mạch đóng cắt ngoài, không cấp nguồn trực tiếp.

Trình tự vào HVSP

Code thực hiện:

```
Tắt VTarget  
Tắt VPP 12 V  
SDI = 0  
SII = 0  
SCI = 0  
Bật VTarget 5 V  
Chờ resetDelay  
Bật 12 V vào RESET  
Giữ tín hiệu Prog_enable  
Phát các xung latch/synchronization  
Chờ trước khi gửi lệnh HVSP
```

ATtiny25/45/85 yêu cầu các chân Prog_enable ở 000, cấp VCC, chờ khoảng 20–60 μ s, áp 11,5–12,5 V vào RESET, giữ trạng thái ít nhất 10 μ s và chờ ít nhất 300 μ s trước khi truyền lệnh.  Microc...

Truyền frame HVSP

Mỗi lần truyền gửi đồng thời:

- 8 bit SDI.
- 8 bit SII instruction.
- Đọc 8 bit SDO.
- Xung SCI.
- Bit bắt đầu và hai bit kết thúc.

Hàm chính:

C

```
static uint8_t transfer(uint8_t data, uint8_t instruction);
```

Tốc độ hiện dùng:

SCI high: 1 μ s

SCI low: 1 μ s

Tương đương khoảng 500 kHz, thấp hơn giới hạn tối đa; datasheet
ATtiny25/45/85 yêu cầu mỗi mức SCI tối thiểu 125 ns.  Microc...

Đã triển khai

Chip erase

C

```
load_command(0x80);  
  
transfer(0x00, 0x64);  
transfer(0x00, 0x6C);  
wait_ready();
```

Ghi Flash

Load command 0x10
Load address thấp
Load address cao
Load byte thấp
Latch byte thấp
Load byte cao
Latch byte cao
Write page
Poll SDO

Hỗ trợ:

- Byte mode.
- Page mode.
- Bit yêu cầu ghi page.
- Địa chỉ word.
- Page lớn được chia thành nhiều frame STK500v2.

Đọc Flash

Load command 0x02
Load address
Read low byte
Read high byte

Ghi và đọc EEPROM

Có:

- EEPROM byte programming.
- EEPROM page programming khi mode yêu cầu.
- Poll SDO ready/busy.
- Tự tăng địa chỉ EEPROM.

Một số ATtiny không hỗ trợ EEPROM page write trong HVSP; trong trường hợp đó AVR Studio phải gửi mode byte programming phù hợp theo mô tả thiết bị. Datasheet ATtiny25/45/85 cũng ghi rõ một số chế độ EEPROM chỉ có trong SPI programming.  Microc...

Fuse

Hỗ trợ:

Address 0 → Low fuse
Address 1 → High fuse
Address 2 → Extended fuse

Có thể dùng để khôi phục chip khi đã đặt:

- `RSTDISBL` .
- `DWEN` .
- Fuse clock sai.
- SPI programming bị vô hiệu hóa.

High-voltage programming đặc biệt hữu ích để khôi phục chip khi chân RESET hoặc giao diện ISP không còn hoạt động bình thường.  Microchip Su... +1

Lock bits, signature và OSCCAL

Đã có:

C

```
AVRHvsp_ProgramLock()  
AVRHvsp_ReadLock()  
AVRHvsp_ReadSignature()  
AVRHvsp_ReadOscCal()
```

Phần cứng bắt buộc

Không nối PA8 vào RESET ATtiny trực tiếp:

PA8
|
điện trở

```
|  
transistor/MOSFET điều khiển  
|  
mạch chuyển 12 V  
|  
RESET ATtiny
```

Nên có:

- Công tắc nguồn target 5 V.
- Công tắc VPP 12 V.
- Buffer hoặc level shifter 3,3/5 V.
- Điện trở nối tiếp khoảng 100–330 Ω trên SDI, SII và SCI.
- Điện trở kéo xuống ở tín hiệu điều khiển VPP để 12 V luôn tắt lúc STM32 reset.
- Bảo vệ không cho 5 V và 12 V đấu ngược trở lại STM32.

Tình trạng kiểm thử

Mã nguồn đã được ghép hoàn chỉnh ở mức firmware và giao thức STK500v2, nhưng tôi chưa thể xác nhận đã chạy thực tế trên phần cứng với từng model ATtiny. Trước tiên nên thử theo thứ tự:

```
Read signature  
Read fuse  
Chip erase  
Read fuse lần nữa  
Ghi một chương trình nhỏ  
Verify Flash
```

Không nên thử ghi fuse ngay lần đầu trước khi việc đọc signature và Flash đã ổn định.

Hết chưa còn vấn đề nào nữa ko

Worked for 32s

Chưa hết. Bộ code hiện tại mới ở mức firmware thử nghiệm, chưa thể coi là mạch nạp STK500v2 hoàn chỉnh hay tương thích hoàn toàn với AVR Studio.

Các vấn đề còn lại quan trọng nhất:

- 1. Chưa kiểm thử bằng phần cứng thật.** Cần thử ít nhất ISP trên ATmega328P/ATmega2560, HVPP trên một ATmega hỗ trợ Parallel

Programming và HVSP trên ATtiny13A/25/45/85. AVR068 chỉ quy định giao thức giữa máy tính và programmer; thuật toán điện áp, timing và lệnh lập trình còn phụ thuộc datasheet từng chip.  Microchip +2

- HVPP chưa sử dụng Control Stack đúng nghĩa.** Code hiện nhận 32 byte `CMD_SET_CONTROL_STACK`, nhưng chủ yếu chỉ lưu lại rồi sử dụng sơ đồ chân cố định. STK500 thật dùng Control Stack để điều khiển ma trận chân socket cho nhiều loại AVR. Vì vậy adapter hiện tại không tự tương thích mọi DIP-20, DIP-28 và DIP-40.  Microchip +1
- HVSP chưa thể áp dụng chung cho mọi ATtiny.** ATtiny25/45/85 có bảng lệnh HVSP riêng; ATtiny24/44/84 cũng hỗ trợ HVSP nhưng timing và bảng lệnh phải kiểm tra theo datasheet tương ứng. ATtiny thế hệ mới dùng UPDI thì firmware này không lập trình được bằng HVSP.  Microchip +2
- Định dạng một số handler HVPP/HVSP cần đối chiếu lại AVR068.** Các trường như mode, delay, poll timeout, address, page size và dữ liệu trong từng command không giống nhau hoàn toàn. Code hiện đã đơn giản hóa một số command thành:

```
C
length
mode
timeout
data[]
```

Trong khi AVR068 có cấu trúc riêng cho từng lệnh PP/HVSP. Nếu AVR Studio gửi đúng format chuẩn nhưng khác giả định của code, dữ liệu sẽ bị đọc sai vị trí.

 Microchip

- Trình tự vào HVSP chưa xác nhận được target thật sự đã vào programming mode.** Hàm hiện tại gần như luôn:

```
C
in_progmode = 1u;
return true;
```

Nó không thực hiện một phép đọc signature hoặc kiểm tra phản hồi hợp lệ trước khi báo `STATUS_CMD_OK`.

- Polling SDO của HVSP còn quá đơn giản.** Code chỉ chờ:

```
C
if (sdo())
    return HVSP_OK;
```

Cần bảo đảm SDO đang ở đúng giai đoạn ready/busy, không phải đang nổi, bị pull-up hoặc còn nổi sai hướng qua level shifter.

7. HVPP RDY/BSY cũng cần xử lý cực tính theo từng target. Không nên mặc định mọi chip đều có cùng trạng thái logic và cùng timing sau Chip Erase, Flash Page Write, EEPROM Write và fuse write.

8. Điều khiển nguồn chưa có phản hồi thật. Các chân `VTARGET_EN` và `VPP12_EN` chỉ bật/tắt GPIO. Firmware chưa đo:

- VTarget thực tế.
- VPP 12 V thực tế.
- Quá dòng/chập mạch.
- VPP có giảm về mức an toàn trước khi cấp VTarget hay không.

Trong khi HV programming yêu cầu điện áp và trình tự nguồn đúng theo datasheet.  Microchip +1

9. Mức logic 3,3 V/5 V chưa được giải quyết bằng code. Mạch cần buffer phù hợp:

- SDI, SII, SCI: STM32 sang AVR.
- SDO: AVR sang STM32.
- Bus HVPP DATA0-DATA7: hai chiều.
- Không dùng level shifter MOSFET I²C thông thường cho các đường push-pull tốc độ cao.
- Không được để 5 V hoặc 12 V quay về STM32.

10. Chưa có interlock chống bật sai nguồn. Cần phần cứng và firmware bảo đảm:

- Không bật VPP nếu VTarget chưa ổn định.
- Không bật ISP và HV cùng lúc.
- Khi STM32 reset, VPP mặc định phải tắt bằng điện trở phần cứng.
- USB ngắt hoặc firmware treo thì 12 V phải tự ngắt.

11. USB TX queue có thể mất frame. Khi queue đầy, hàm hiện tăng biến `dropped` rồi bỏ frame. Với STK500v2, mất một response có thể khiến AVR Studio timeout và mất đồng bộ. Cần dành riêng buffer phản hồi hoặc không nhận command mới khi queue chưa còn chỗ.

12. Ring buffer nhận cũng có thể tràn. Parser có reset khi overflow nhưng không chắc AVR Studio sẽ tự đồng bộ ngay. Nên trả lỗi hoặc dừng endpoint tạm thời thay vì âm thầm bỏ dữ liệu.

13. Chưa xử lý sequence trùng/lặp. Khi máy tính retry cùng một sequence vì timeout, programmer không nên vô tình thực hiện lại lệnh phá huỷ như chip erase hoặc ghi fuse mà không có chiến lược rõ ràng.

14. Chưa có timeout tổng và cơ chế fail-safe. Nếu đang ở HV mode mà PC dừng giao tiếp, firmware có thể giữ target ở chế độ nguồn/VPP lâu hơn cần

thiết. Cần watchdog phiên lập trình, ví dụ không nhận command trong một khoảng thời gian thì:

C

```
VPP_OFF();  
VTARGET_OFF();  
bus_high_impedance();
```

15. Chưa có project CubeIDE hoàn chỉnh. ZIP chủ yếu có các file nguồn và đoạn patch, chưa có đầy đủ:

- .ioc
- startup code
- linker script
- USB descriptors đã cấu hình
- file project CubeIDE
- cấu hình GPIO hoàn chỉnh
- sơ đồ mạch nguồn 5 V/12 V và level shifting

16. Khả năng nhận diện trong AVR Studio chưa được xác nhận. Trả chuỗi STK500_2 chưa đủ để chứng minh toàn bộ chức năng ISP/HVPP/HVSP xuất hiện và hoạt động đúng trong AVR Studio. Cần thu log từng frame AVR Studio gửi và so sánh với AVR068.

Kết luận

Hiện tại có thể đánh giá:

Phần	Tình trạng
Parser frame STK500v2	Có nền tảng
ISP cơ bản	Gần dùng thử được
Flash/EEPROM ISP	Cần test và sửa polling
HVPP	Chưa hoàn chỉnh
HVSP	Chưa hoàn chỉnh
Tương thích mọi ATmega/ATtiny	Chưa
An toàn nguồn 12 V	Chưa đủ
Project build/nạp trực tiếp	Chưa đầy đủ

Phần	Tình trạng
Xác nhận bằng AVR Studio thật	Chưa

Vì vậy **không nên chế tạo PCB dựa hoàn toàn vào phiên bản hiện tại**. Bước tiếp theo đúng là hoàn thiện một target cụ thể trước, chẳng hạn:

```
ISP:  ATmega328P
HVPP: ATmega328P DIP-28
HVSP: ATtiny85 DIP-8
```

Sau khi ba chip này vượt qua toàn bộ chu trình `signature → erase → write → verify → fuse → power-cycle`, mới mở rộng bảng thiết bị sang các ATmega/ATtiny khác.

Vấn đề 12v hv giải quyết phần cứng thế nào

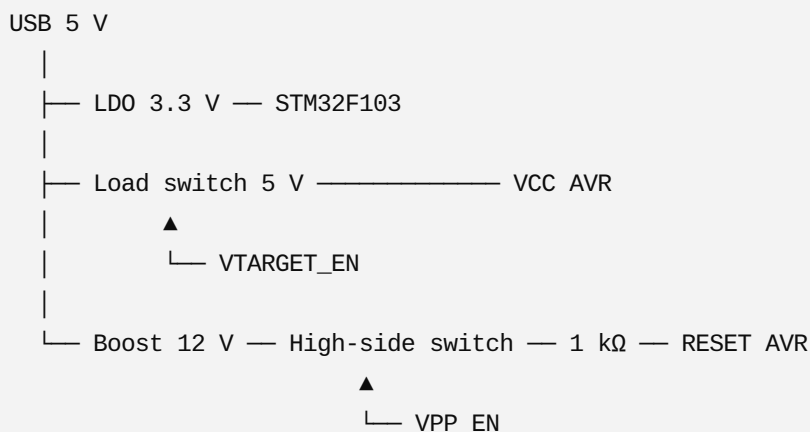
Giải pháp phần cứng đúng là **không tạo 12 V trực tiếp từ STM32**. STM32 chỉ điều khiển hai công tắc điện tử:

- Công tắc cấp **VTarget 5 V** cho AVR.
- Công tắc đưa **VPP khoảng 12 V** vào chân RESET AVR.

HVPP và HVSP đều cần điện áp cao khoảng 12 V trên RESET; riêng ATtiny25/45/85 yêu cầu trình tự cấp VCC rồi đưa khoảng 11,5–12,5 V vào RESET.

 Micro... +1

1. Sơ đồ khối đề nghị



STM32 tuyệt đối không được tiếp xúc trực tiếp với đường 12 V.

2. Mạch tạo 12 V

Có thể dùng boost converter từ USB 5 V lên 12 V:

USB_5V — Boost converter — VPP_12V

Các IC phù hợp về nguyên lý:

- MT3608.
- TPS61040.
- TPS61170.
- Một module boost 5 V → 12 V ổn định.

Dòng VPP yêu cầu không lớn, nhưng nên thiết kế nguồn khoảng:

12 V, tối thiểu 20–50 mA

Đặt:

VPP_12V — 10 μ F
└─ 100 nF

ngay gần công tắc VPP.

Nên đặt điện áp khoảng:

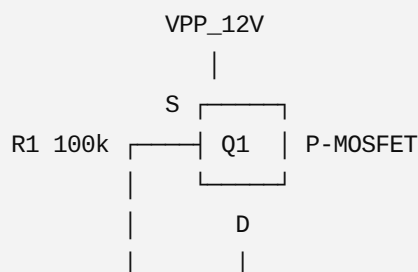
12.0 V

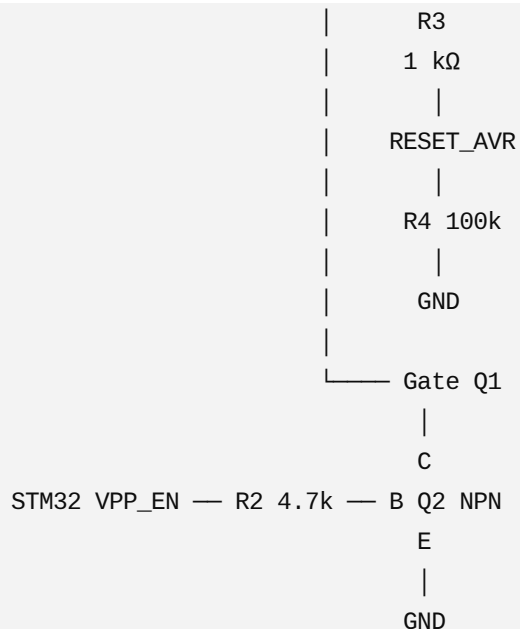
Không chỉnh lên 13–14 V. Datasheet ATtiny25/45/85 nêu vùng điện áp high-voltage programming khoảng 11,5–12,5 V.  Microc...

3. Công tắc đưa 12 V vào RESET

Giải pháp dễ làm là dùng **P-channel MOSFET high-side**, điều khiển qua transistor NPN.

Sơ đồ





Linh kiện:

Q1: P-MOSFET chịu tối thiểu 20-30 V
 Q2: BC817, MMBT3904 hoặc 2N3904
 R1: 100 kΩ
 R2: 4.7 kΩ
 R3: 470 Ω - 1 kΩ
 R4: 47 kΩ - 100 kΩ

Một số P-MOSFET có thể dùng:

A03401A
 DMP3010
 BSS84

BSS84 có điện trở dẫn cao hơn nhưng dòng VPP nhỏ nên vẫn có thể dùng.

Trạng thái hoạt động

VPP_EN = 0

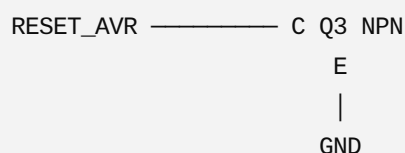
Q2 tắt, R1 kéo gate Q1 lên 12 V:

VGS(Q1) = 0 V
 Q1 tắt
 RESET không có 12 V

VPP_EN = 1

Q2 dẫn, kéo gate Q1 xuống gần GND:

12 V được đưa tới RESET AVR

$$|V_{GS}|_{\max} \geq 20 \text{ V}$$


Điều khiển Q3 bằng tín hiệu đảo với `VPP_EN` , hoặc đơn giản dùng điện trở 47 kΩ nếu tốc độ xả đo thực tế đạt yêu cầu.

Giải pháp an toàn hơn:

VPP bật → Q1 bật, Q3 tắt
VPP tắt → Q1 tắt, Q3 bật kéo RESET xuống

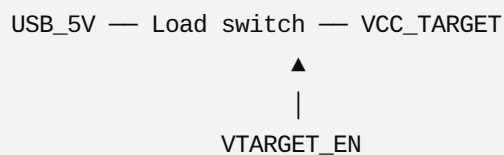
Có thể dùng một cổng logic `74LVC1G04` để tạo tín hiệu đảo.

Nhưng phải tránh Q1 và Q3 cùng dẫn, vì khi đó 12 V bị chập xuống GND. Firmware hoặc mạch logic nên tạo dead-time vài microsecond.

6. Công tắc nguồn VTarget 5 V

Không nên cấp VCC AVR trực tiếp từ GPIO STM32.

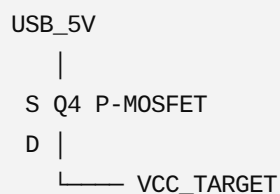
Dùng load switch:



IC phù hợp:

TPS22910A
TPS22918
AP22802
SY6280

Hoặc dùng P-MOSFET high-side tương tự:



Nên thêm:

VCC_TARGET — 10 μF — GND
VCC_TARGET — 100 nF — GND

và điện trở xả:

VCC_TARGET — 10 kΩ — GND

Điện trở 10 kΩ giúp VTarget tắt thật sự giữa các chu kỳ vào HV mode.

7. Interlock chống bật 12 V sai thời điểm

Không nên chỉ dựa vào firmware.

Có thể tạo điều kiện:

VPP_SWITCH_ENABLE = VPP_EN AND VTARGET_GOOD

Dùng một cổng AND:

74LVC1G08

Sơ đồ:

STM32 VPP_EN ————┐
AND — điều khiển Q2
VTarget Power Good ———┘

Nếu load switch không có Power Good , có thể dùng:

- Comparator đo VTarget.
- Hoặc một tín hiệu VTARGET_EN đã qua delay RC.

Cách đơn giản:

VPP_EN thực tế = MCU_VPP_EN AND MCU_VTARGET_EN

Như vậy STM32 không thể bật VPP nếu chưa bật VTarget.

8. Mạch đo VPP và VTarget

Nên cho STM32 đo cả 5 V và 12 V bằng ADC.

Đo VPP 12 V

Dùng bộ chia:

VPP_12V — 100 kΩ —┬— ADC_VPP
|

27 kΩ

|
GND

Ở 12,5 V:

$$V_{ADC} = 12.5 \times 27 / (100 + 27) \\ \approx 2.66 \text{ V}$$

An toàn cho ADC 3,3 V.

Thêm:

ADC_VPP — 10 nF — GND
ADC_VPP — 1 kΩ — chân ADC STM32

Đo VTarget 5 V

VCC_TARGET — 47 kΩ — ADC_VTARGET
|
47 kΩ
|
GND

Ở 5 V:

$$V_{ADC} = 2.5 \text{ V}$$

Firmware chỉ bật VPP nếu:

C

4.5 V ≤ VTarget ≤ 5.5 V
11.5 V ≤ VPP ≤ 12.5 V

Các giới hạn thực tế phải lấy theo datasheet của chip được chọn. Ví dụ ATtiny25/45/85 mô tả VCC khoảng 4,5–5,5 V và RESET high-voltage khoảng 11,5–12,5 V trong trình tự HVSP.  Microc...

9. Bảo vệ quá dòng nguồn target

Nên dùng load switch có giới hạn dòng hoặc thêm polyfuse:

USB_5V — Polyfuse 100–250 mA — load switch — VCC_TARGET

Hoặc dùng IC có current limit:

SY6280
TPS2553
AP22652

Đặt giới hạn khoảng:

100–200 mA

cho mạch nạp một AVR đơn lẻ.

Nếu target bị cắm ngược hoặc chập, firmware phải:

```
C  
  
VPP_OFF();  
VTARGET_OFF();
```

10. Chuyển mức tín hiệu 3,3 V và 5 V

STM32F103 có nhiều chân được đánh dấu chịu 5 V, nhưng không phải tất cả chân đều chịu 5 V; phải kiểm tra ký hiệu **FT** trong bảng pin của datasheet.

 STMicroelect...

Tuy nhiên với mạch nạp đa năng, không nên phụ thuộc vào khả năng chịu 5 V của từng chân.

HVSP

Các đường STM32 → AVR:

SDI
SII
SCI

Dùng buffer 5 V nhận được mức logic 3,3 V:

74HCT125
74AHCT125

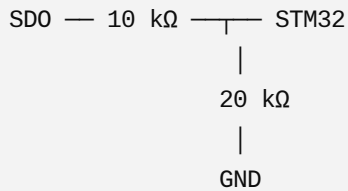
74HCT chạy 5 V nhưng ngưỡng đầu vào phù hợp tín hiệu 3,3 V.

STM32 3.3 V → 74HCT125 → AVR 5 V

Đường AVR → STM32:

SD0 → 74LVC1G125 chạy 3,3 V

hoặc chia áp:



Buffer vẫn tốt hơn chia áp.

HVPP bus DATA0-DATA7

Dùng transceiver hai chiều:

74LVC245

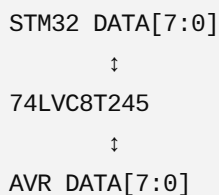
Cấp nguồn 74LVC245 ở 3,3 V và chọn loại đầu vào chịu 5 V phù hợp, hoặc dùng transceiver dual-supply:

74LVC8T245

74AXC8T245

SN74AVC8T245

Sơ đồ:



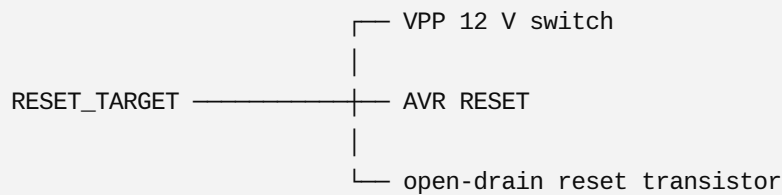
Chân DIR và /OE do firmware điều khiển.

Không dùng module chuyển mức MOSFET BSS138 kiểu I²C cho SCI, SII, SDI hoặc bus HVPP.

11. Không cho 12 V lọt sang đường logic

RESET AVR có thể dùng chung với chân tín hiệu trong một số chế độ. Cần cô lập đường reset logic bình thường khỏi VPP 12 V.

Sơ đồ nên là:



Không nối RESET AVR trực tiếp với chân STM32.

Để kéo RESET xuống khi ISP:

```
RESET_AVR — Collector Q5 NPN
                      Emitter — GND
STM32_RESET_CTRL — 4.7 kΩ — Base
```

Khi HV mode:

- Q5 phải tắt.
- Q1 mới được phép đưa 12 V vào RESET.

Thêm điện trở bảo vệ:

```
Q1 output — 470 Ω hoặc 1 kΩ — RESET AVR
```

12. Trình tự phần cứng an toàn

Vào HVSP/HVPP

1. Tắt VPP 12 V.
2. Tắt VTarget.
3. Đưa tất cả tín hiệu lập trình về mức quy định.
4. Chuyển các bus sang trạng thái an toàn.
5. Bật VTarget 5 V.
6. Đo và xác nhận VTarget ổn định.
7. Chờ theo datasheet.
8. Bật switch VPP 12 V.
9. Đo và xác nhận VPP.
10. Bắt đầu giao tiếp lập trình.

Thoát HVSP/HVPP

1. Dừng các xung lập trình.
2. Tắt VPP 12 V.
3. Chờ VPP giảm xuống dưới khoảng 1 V.

4. Tắt VTarget.
5. Đặt mọi buffer về high-impedance.

Việc cấp 12 V vào RESET là đặc trưng của HVSP/HVPP, nhưng số chân và thuật toán khác nhau giữa hai giao diện.  Microchip De...

13. Mạch đề nghị tối thiểu

Danh sách linh kiện thực tế:

- 1 × boost 5 V → 12 V
- 1 × A03401A P-MOS VPP
- 1 × BC817 điều khiển P-MOS VPP
- 1 × load switch 5 V VTarget
- 1 × 74AHCT125 STM32 → AVR
- 1 × 74LVC125 AVR → STM32
- 1 × 74LVC8T245 bus HVPP
- 1 × comparator hoặc ADC kiểm tra VPP/VTarget
- 1 × polyfuse 100-250 mA
- 2 × LED báo 5 V và 12 V

Giá trị điện trở chính

- Gate pull-up Q1: 100 kΩ
- Base Q2: 4.7 kΩ
- VPP → RESET: 470 Ω - 1 kΩ
- RESET discharge: 47 kΩ
- ADC VPP: 100 kΩ / 27 kΩ
- ADC VTarget: 47 kΩ / 47 kΩ
- Tín hiệu lập trình: 47-220 Ω nối tiếp

Điểm quan trọng nhất là **VPP phải mặc định tắt bằng phần cứng**. Khi STM32 chưa khởi động, bị reset, mất USB hoặc treo chương trình, gate Q1 phải được kéo về source để không thể xuất 12 V.

AVRISP MKII so sánh với stk500

Nếu chỉ xét **làm mạch nạp AVR bằng ISP**, thì **AVRISP mkII tốt hơn STK500**.

Nếu xét **thiết bị phát triển đa năng**, thì **STK500 mạnh hơn rất nhiều**.

So sánh chi tiết:

Tiêu chí	AVRISP mkII	STK500
ISP SPI	✓	✓
HVPP	✗	✓
HVSP	✗	✓
Cấp nguồn target	Có (giới hạn)	Có, điều chỉnh được
Điều chỉnh ISP Clock	Có	Có
Socket DIP	✗	✓
Đọc/Ghi Fuse	✓	✓
Đọc/Ghi Lock	✓	✓
Chip Erase	✓	✓
Debug	✗	✗ (STK500 không phải debugger)
USB	USB	RS232 (bản gốc)

AVRISP mkII

Mục tiêu của AVRISP mkII chỉ là:

USB



ISP SPI



AVR

Nó **không có HVPP/HVSP**.

Nếu:

- tắt SPIEN
- tắt RESET
- fuse sai

thì AVRISP mkII **bó tay**.

Muốn cứu chip phải dùng:

- STK500
- AVR Dragon
- hoặc mạch HV tự làm.

STK500

STK500 là một hệ thống phát triển hoàn chỉnh.

Nó có:

ISP

HVPP

HVSP

Socket DIP

nguồn target

clock target

đo VTarget

control stack

routing matrix

Ví dụ:

ATmega328P bị:

RSTDISBL = 1

AVRISP mkII:

Không kết nối được

STK500:

HVPP

↓

Ghi lại fuse

↓

Chip sống lại

Tốc độ

ISP:

AVRISP mkII

thường nhanh hơn STK500.

STK500 ưu tiên tính linh hoạt hơn.

Firmware

AVRISP mkII firmware đơn giản hơn rất nhiều.

Khoảng:

USB

↓

STK500v2 protocol

↓

SPI ISP

Là xong.

Trong khi STK500 còn phải quản lý:

Control Stack

Voltage

Clock

Routing

HVPP

HVSP

Socket

LED

ADC

VTarget

...

Firmware lớn hơn nhiều.

Phần cứng

AVRISP mkII:

USB MCU

↓

Level shifter

↓

ISP connector

Chỉ vài chục linh kiện.

STK500:

Có hàng trăm linh kiện.

Ví dụ:

- DAC điều chỉnh VTarget
- nguồn
- boost HV
- analog switch
- mux
- driver
- socket
- oscillator

- RS232
- ADC
- control stack

Board lớn gấp nhiều lần.

Giao thức PC

Đây là điểm nhiều người nhầm.

AVRISP mkII và STK500 đều dùng giao thức STK500v2 với AVR Studio.

Khác nhau là firmware.

Ví dụ:

AVR Studio gửi:

```
CMD_SIGN_ON
```

```
CMD_ENTER_PROGMODE_ISP
```

```
CMD_PROGRAM_FLASH_ISP
```

Cả hai đều hiểu.

Nhưng nếu gửi:

```
CMD_ENTER_PROGMODE_PP
```

thì

AVRISP mkII trả:

```
Unknown command
```

còn

STK500 sẽ chuyển sang phần cứng HVPP.

Nếu mục tiêu của bạn

Qua các cuộc trao đổi trước, mình thấy bạn đang muốn:

- dùng **STM32F103**
- tương thích **AVR Studio**
- nạp **mọi AVR**
- cứu fuse
- HVPP
- HVSP

thì **không nên mô phỏng AVRISP mkII**.

Nên mô phỏng **STK500 đầy đủ** hoặc thậm chí **AVR Dragon**, vì:

- STK500 đã có toàn bộ giao thức cho ISP, HVPP và HVSP.
- AVR Dragon còn hỗ trợ thêm các giao diện như JTAG và debugWIRE (tùy dòng AVR), nhưng phần cứng và firmware phức tạp hơn.

Với mục tiêu của bạn, STK500 là nền tảng hợp lý hơn vì nó bao phủ gần như toàn bộ các chức năng lập trình AVR cổ điển mà bạn đang hướng tới.

vay full code AVRISP MKII đi

Worked for 38s

Đã làm bộ mã nguồn **AVRISP mkII clone cho STM32F103C8**, giao tiếp trực tiếp bằng **USB Bulk**, không dùng cổng COM:

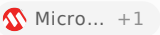
Download STM32F103 AVRISP mkII ISP Full

Nội dung đã hoàn thành

- USB Vendor-Specific class.
- Bulk endpoint:
 - 0x81 : IN.
 - 0x02 : OUT.
 - Packet tối đa 64 byte.
- USB VID/PID theo descriptor AVRISP mkII:
 - VID 0x03EB .
 - PID 0x2104 .
- Chuỗi nhận dạng:

C

"AVRISP_MK2"

- Đọc điện áp VTarget bằng ADC.
- Điều chỉnh ISP SCK từ tốc độ cao xuống khoảng 50 Hz.
- SPI phần cứng ở tốc độ cao.
- Bit-bang ở tốc độ thấp.
- Enter/Leave ISP programming.
- Chip erase.
- Đọc/ghi Flash.
- Flash lớn và Load Extended Address.
- Đọc/ghi EEPROM.
- Đọc/ghi fuse.
- Đọc/ghi lock bit.
- Đọc signature.
- Đọc OSCCAL.
- `CMD_SPI_MULTI`.
- USB RX/TX queue không chặn.
- Target phải có nguồn riêng, đúng với AVRISP mkII thật: chân VTarget chủ yếu dùng để cảm nhận điện áp, không cấp nguồn cho target. 

Cấu trúc USB khác STK500

STK500 qua COM dùng frame:

1B SEQUENCE SIZE 0E PAYLOAD CHECKSUM

AVRISP mkII không dùng frame đó. Mỗi USB message có dạng:

COMMAND_ID + COMMAND_DATA

Một command có thể được chia thành nhiều packet USB 64 byte. Packet ngắn hơn 64 byte hoặc packet ZLP đánh dấu kết thúc message. Đây là cấu trúc được mô tả trong AVR069. 

Chân STM32 mặc định

STM32F103

AVR ISP

PA5

SCK

PA6

MISO

PA7

MOSI

PB0

RESET

STM32F103

AVR ISP

PA0 ADC

VTarget sense

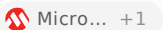
GND

GND

PB0 nên điều khiển RESET qua transistor open-drain, không nối theo kiểu push-pull trực tiếp khi muốn hỗ trợ nhiều mức VTarget.

Giới hạn của bản này

Đây là bản đầy đủ cho **AVRISP mkII ở chế độ ISP/SPI**.

AVRISP mkII chính hãng còn hỗ trợ PDI và TPI ngoài ISP. Phần PDI/TPI chưa có trong ZIP này, vì chúng dùng giao thức vật lý và nhóm command khác hoàn toàn. 

Vì vậy:

Giao diện

Trạng thái

ISP cho ATmega/ATtiny cổ điển

Có

Flash/EEPROM/Fuse/Lock

Có

ATmega2560 extended address

Có

PDI cho XMEGA

Chưa

TPI cho một số ATtiny

Chưa

UPDI cho tinyAVR/megaAVR mới

Không thuộc AVRISP mkII ISP

HVPP/HVSP

Không thuộc AVRISP mkII

Trong ZIP có mã nguồn, USB class, descriptor patch, phần thêm vào `main.c`, cấu hình CubeMX và tài liệu kiểm tra. Mã đã được kiểm tra cân bằng cú pháp cơ bản, nhưng vẫn cần build bằng đúng phiên bản STM32CubeF1 và kiểm thử thực tế trên phần cứng trước khi ghi fuse. 

GIAI THÍCH CHI TIẾT DE TOI HIEU CODE

Worked for 20s

Dưới đây là cách hiểu toàn bộ code **AVRISP mkII** trên **STM32F103** theo thứ tự dữ liệu thực sự chạy trong mạch.

1. Sơ đồ hoạt động tổng thể

AVR Studio / Atmel Studio

|
| USB Bulk
▼

```
| usbd_avrisp.c  
| Nhận và gửi packet USB
```

|
▼

```
| avrisp_mkii.c  
| Giải mã command AVR069  
| Flash, EEPROM, Fuse...
```

|
▼

```
| avr_isp_phy.c  
| Điều khiển SPI và RESET  
| Hardware SPI / bit-bang
```

|
▼

Vi điều khiển AVR

Ba file quan trọng nhất:

```
USB_DEVICE/App/usbd_avrisp.c  
Core/Src/avrisp_mkii.c  
Core/Src/avr_isp_phy.c
```

2. Vì sao AVRISP mkII không dùng cổng COM?

STK500 qua cổng nối tiếp có frame:

```
1B SEQUENCE SIZE_H SIZE_L 0E PAYLOAD CHECKSUM
```

Nhưng AVRISP mkII không dùng frame trên.

Nó sử dụng USB Vendor-Specific với:

```
Bulk IN  = endpoint 0x81
Bulk OUT = endpoint 0x02
```

Máy tính gửi trực tiếp:

```
COMMAND_ID + COMMAND_DATA
```

Ví dụ yêu cầu nhận dạng có thể chỉ gồm:

```
01
```

Trong đó:

```
C
#define CMD_SIGN_ON 0x01u
```

STM32 trả về:

```
01 00 0A "AVRISP_MK2"
```

Ý nghĩa:

```
01  CMD_SIGN_ON
00  STATUS_CMD_OK
0A  Chuỗi dài 10 byte
...  AVRISP_MK2
```

3. File `usbd_avrISP.h`

File này định nghĩa endpoint USB:

```
C
#define AVRISP_IN_EP  0x81U
#define AVRISP_OUT_EP 0x02U
```

Ý nghĩa `0x81`

Bit cao bằng 1:

0x81 = IN endpoint

Dữ liệu đi:

STM32 → máy tính

Số endpoint là:

0x81 & 0x7F = 1

Ý nghĩa 0x02

Bit cao bằng 0:

0x02 = OUT endpoint

Dữ liệu đi:

Máy tính → STM32

Kích thước packet:

C

```
#define AVRISP_FS_MAX_PACKET_SIZE 64U
```

USB Full Speed Bulk endpoint dùng packet tối đa 64 byte.

4. File usbd_avrISP.c

Đây là USB class tự viết để thay cho USB CDC.

4.1 Cấu trúc quản lý USB

C

```
typedef struct {  
    uint8_t rx_buf[AVRISP_FS_MAX_PACKET_SIZE];  
    uint8_t *tx_buf;  
    uint16_t tx_len;  
} USBD_AVRISP_HandleTypeDef;
```

Nó chứa:

- `rx_buf` : vùng nhận một packet USB, tối đa 64 byte.
- `tx_buf` : con trỏ đến dữ liệu chuẩn bị gửi.
- `tx_len` : số byte cần gửi.

4.2 Descriptor configuration

C

```
static uint8_t USBD_AVRISP_CfgFSDesc[]
```

Descriptor mô tả thiết bị có:

1 interface
2 endpoint
Vendor-specific class

Phần interface:

C

```
0x09, USB_DESC_TYPE_INTERFACE,  
0x00, 0x00, 0x02,  
0xFF, 0x00, 0x00, 0x00,
```

Ý nghĩa:

Byte	Ý nghĩa
0x00	Interface số 0
0x02	Có hai endpoint
0xFF	Vendor-specific class

Nó không phải:

- CDC
- HID
- Mass Storage

AVR Studio nhận nó qua VID/PID và driver AVRISP mkII.

4.3 Bulk IN endpoint

C

```
0x07, USB_DESC_TYPE_ENDPOINT,  
AVRISP_IN_EP, 0x02,  
64, 0,  
0x0A
```

Trong đó:

```
AVRISP_IN_EP = 0x81  
0x02         = Bulk endpoint  
64           = max packet size
```

4.4 Bulk OUT endpoint

C

```
0x07, USB_DESC_TYPE_ENDPOINT,  
AVRISP_OUT_EP, 0x02,  
64, 0,  
0x0A
```

Máy tính gửi command vào endpoint này.

5. Khởi tạo USB class

Hàm:

C

```
static uint8_t USBD_AVRISP_Init(  
    USBD_HandleTypeDef *pdev,  
    uint8_t cfgidx)
```

Mở hai endpoint:

C

```
USB_LL_OpenEP(  
    pdev,  
    AVRISP_IN_EP,  
    USB_EP_TYPE_BULK,  
    AVRISP_FS_MAX_PACKET_SIZE  
);
```

và:

C

```
USBDD_LL_OpenEP(  
    pdev,  
    AVRISP_OUT_EP,  
    USBDD_EP_TYPE_BULK,  
    AVRISP_FS_MAX_PACKET_SIZE  
);
```

Sau đó cấp RAM cho class:

C

```
pdev->pClassData =  
    USBDD_malloc(sizeof(USBDD_AVRISP_HandleTypeDef));
```

Cuối cùng chuẩn bị nhận packet đầu tiên:

C

```
USBDD_LL_PrepareReceive(  
    pdev,  
    AVRISP_OUT_EP,  
    h->rx_buf,  
    AVRISP_FS_MAX_PACKET_SIZE  
);
```

Nếu thiếu `PrepareReceive()` , endpoint OUT không nhận dữ liệu.

6. Khi máy tính gửi dữ liệu

Callback:

C

```
static uint8_t USBDD_AVRISP_DataOut(  
    USBDD_HandleTypeDef *pdev,  
    uint8_t epnum)
```

Lấy số byte thật đã nhận:

C

```
uint16_t len =
```

Chuyển packet sang lớp protocol:

C

```
AVRISP_MKII_USB_RxPacket(  
    h->rx_buf,  
    len,  
    len < AVRISP_FS_MAX_PACKET_SIZE  
);
```

Tham số cuối:

C

```
len < 64
```

cho biết đây có phải packet ngắn hay không.

Packet ngắn thường báo:

Đây là packet cuối của một USB message

Sau đó phải mở lại endpoint:

C

```
USBD_LL_PrepareReceive(  
    pdev,  
    AVRISP_OUT_EP,  
    h->rx_buf,  
    64  
);
```

7. Một command có thể gồm nhiều packet USB

Ví dụ AVR Studio gửi command ghi Flash dài 266 byte.

USB chia thành:

```
Packet 1: 64 byte  
Packet 2: 64 byte  
Packet 3: 64 byte  
Packet 4: 64 byte  
Packet 5: 10 byte
```

Hàm:

C

```
AVRISP_MKII_USB_RxPacket()
```

ghép các packet lại trong:

C

```
static uint8_t assembling[AVRISP_USB_MESSAGE_MAX];  
static uint16_t assembling_len;
```

Mỗi packet được nối vào:

C

```
memcpy(  
    &assembling[assembling_len],  
    data,  
    length  
);  
  
assembling_len += length;
```

Khi gặp packet ngắn:

C

```
if (short_packet)
```

message hoàn chỉnh được đưa vào hàng đợi:

C

```
rxq[rx_w]
```

Sau đó reset bộ ghép:

C

```
assembling_len = 0u;  
rx_overflow = 0u;
```

8. Hàng đợi nhận USB

Cấu trúc một message:

C

```
typedef struct {
    uint8_t data[AVRISP_USB_MESSAGE_MAX];
    uint16_t len;
} usb_message_t;
```

Mỗi phần tử chứa:

- toàn bộ command;
- chiều dài command.

Hàng đợi nhận:

C

```
#define RX_QUEUE_DEPTH 3u

static usb_message_t rxq[RX_QUEUE_DEPTH];
static volatile uint8_t rx_r;
static volatile uint8_t rx_w;
static volatile uint8_t rx_count;
```

Ý nghĩa:

- `rx_w` : vị trí USB callback ghi command mới.
- `rx_r` : vị trí main loop lấy command.
- `rx_count` : số command đang chờ.

USB callback chỉ nhận và xếp hàng. Nó không trực tiếp ghi Flash AVR.

Điều này tốt vì callback USB cần chạy nhanh.

9. Hàng đợi gửi USB

Tương tự:

C

```
#define TX_QUEUE_DEPTH 3u

static usb_message_t txq[TX_QUEUE_DEPTH];
static volatile uint8_t tx_r;
static volatile uint8_t tx_w;
static volatile uint8_t tx_count;
static volatile uint8_t tx_busy;
```

Hàm:

C

```
static void tx_enqueue(  
    const uint8_t *data,  
    uint16_t len)
```

sao chép response vào hàng đợi.

Hàm:

C

```
static void tx_task(void)
```

chỉ gửi khi:

C

```
tx_busy == 0
```

và có dữ liệu:

C

```
tx_count > 0
```

Gửi bằng:

C

```
USBDAVRISP_TransmitPacket(  
    &hUsbDeviceFS,  
    txq[tx_r].data,  
    txq[tx_r].len  
);
```

10. Khi gửi USB hoàn tất

USB gọi:

C

```
USBDAVRISP_DataIn()
```


Sau đó:

C

```
AVRISP_MKII_USB_TxComplete();
```

Hàm này bỏ message vừa gửi khỏi queue:

C

```
tx_r = (tx_r + 1u) % TX_QUEUE_DEPTH;  
--tx_count;  
tx_busy = 0u;
```

Lần gọi `AVRISP_MKII_Task()` tiếp theo sẽ gửi response kế tiếp.

11. Hàm chính `AVRISP_MKII_Task()`

Trong `main.c` :

C

```
while (1)  
{  
    AVRISP_MKII_Task();  
}
```

Hàm thực hiện:

C

```
void AVRISP_MKII_Task(void)  
{  
    AVRISP_PHY_Task();  
  
    if (rx_count)  
    {  
        ...  
        dispatch(msg.data, msg.len);  
    }  
  
    tx_task();  
}
```

Trình tự:

1. Chạy công việc lớp vật lý.

2. Nếu có command USB thì lấy ra.
 3. Gọi `dispatch()`.
 4. Gửi response đang chờ.
-

12. Hàm `dispatch()`

Đây là bộ phân phối command chính.

C

```
static void dispatch(  
    const uint8_t *p,  
    uint16_t n)
```

Trong đó:

- `p`: toàn bộ message.
- `n`: số byte message.
- `p[0]`: mã command.

Ví dụ:

C

```
switch (p[0])  
{  
    case CMD_SIGN_ON:  
        cmd_sign_on();  
        break;  
  
    case CMD_SET_PARAMETER:  
        ...  
        break;  
  
    case CMD_ENTER_PROGMODE_ISP:  
        cmd_enter(p,n);  
        break;  
}
```

13. Các command AVRISP mkII

C

```
#define CMD_SIGN_ON          0x01u  
#define CMD_SET_PARAMETER    0x02u
```

```

#define CMD_GET_PARAMETER      0x03u
#define CMD_LOAD_ADDRESS      0x06u

#define CMD_ENTER_PROGMODE_ISP 0x10u
#define CMD_LEAVE_PROGMODE_ISP 0x11u
#define CMD_CHIP_ERASE_ISP     0x12u
#define CMD_PROGRAM_FLASH_ISP  0x13u
#define CMD_READ_FLASH_ISP     0x14u
#define CMD_PROGRAM_EEPROM_ISP 0x15u
#define CMD_READ_EEPROM_ISP    0x16u

```

Những command phía sau:

C

```

CMD_PROGRAM_FUSE_ISP
CMD_READ_FUSE_ISP
CMD_PROGRAM_LOCK_ISP
CMD_READ_LOCK_ISP
CMD_READ_SIGNATURE_ISP
CMD_READ_OSCCAL_ISP
CMD_SPI_MULTI

```

14. CMD_SIGN_ON

Hàm:

C

```

static void cmd_sign_on(void)
{
    static const uint8_t sig[] = "AVRISP_MK2";

    response[0] = CMD_SIGN_ON;
    response[1] = STATUS_CMD_OK;
    response[2] = 10u;

    memcpy(&response[3], sig, 10u);

    tx_enqueue(response, 13u);
}

```

Response:

```

Byte 0: 0x01
Byte 1: 0x00

```

STATUS_CMD_OK :

C

```
#define STATUS_CMD_OK 0x00u
```

15. Parameter của programmer

Một số parameter:

C

```
#define PARAM_HW_VER          0x90u
#define PARAM_SW_MAJOR        0x91u
#define PARAM_SW_MINOR        0x92u
#define PARAM_VTARGET         0x94u
#define PARAM_SCK_DURATION    0x98u
#define PARAM_RESET_POLARITY  0x9Eu
#define PARAM_STATUS_TGT_CONN 0xA1u
```

Ví dụ đọc VTarget

C

```
case PARAM_VTARGET:
    return (uint8_t)(
        (AVRISP_PHY_ReadTargetMillivolts() + 50u) / 100u
    );
```

Nếu ADC đo được:

5000 mV

thì trả:

5000 / 100 = 50

Nghĩa là:

5.0 V

Parameter dùng đơn vị 0,1 V.

16. Thiết lập ISP clock

AVR Studio gửi:

```
CMD_SET_PARAMETER
PARAM_SCK_DURATION
duration
```

Code:

```
C

case PARAM_SCK_DURATION:
    parameters[id] = value;
    AVRISP_PHY_SetSckDuration(value);
    return STATUS_CMD_OK;
```

Hàm thật sự đổi tốc độ nằm trong:

```
avr_isp_phy.c
```

17. File `avr_isp_phy.c`

Đây là lớp tiếp xúc trực tiếp với chân STM32.

Mặc định:

```
PA5 = SCK
PA6 = MISO
PA7 = MOSI
PB0 = RESET
PA0 = ADC VTarget
```

Nó hỗ trợ hai chế độ:

```
SPI hardware
Bit-bang GPIO
```

18. Bảng tốc độ SCK

```
C
```

```
static const uint32_t sck_table[]
```

Mỗi giá trị `PARAM_SCK_DURATION` ánh xạ sang một tần số.

Ví dụ đầu bảng:

C

```
8000000,  
4000000,  
2000000,  
1000000,  
500000,  
250000,  
125000
```

Cuối bảng:

C

```
2000,  
1000,  
500,  
250,  
125,  
100,  
50
```

Như vậy AVR Studio có thể yêu cầu SCK rất chậm khi target dùng clock thấp.

19. Chọn SPI hardware hay bit-bang

C

```
if (g_sck_hz >=  
    (HAL_RCC_GetPCLK2Freq() / 256u))  
{  
    g_use_bitbang = 0u;  
    hardware_spi_mode(g_sck_hz);  
}  
else  
{  
    g_use_bitbang = 1u;  
    ...  
    gpio_spi_mode();  
}
```

Nếu PCLK2 là 72 MHz:

$$72 \text{ MHz} / 256 = 281.25 \text{ kHz}$$

SPI hardware chỉ giảm tối đa đến khoảng 281 kHz.

Do đó:

SCK $\geq$ 281 kHz $\rightarrow$ hardware SPI
SCK < 281 kHz $\rightarrow$ GPIO bit-bang

20. Chọn prescaler SPI hardware

```
C
uint32_t divs[8] = {
    2, 4, 8, 16, 32, 64, 128, 256
};
```

Code chọn tốc độ SPI không vượt quá tốc độ AVR Studio yêu cầu:

```
C
if ((pclk / divs[i]) <= requested_hz)
{
    selected = brs[i];
    break;
}
```

Ví dụ yêu cầu:

SCK = 1 MHz

Với PCLK2 = 72 MHz:

/64 = 1.125 MHz $\rightarrow$ quá cao
/128 = 562.5 kHz $\rightarrow$ không vượt 1 MHz

Code sẽ chọn /128.

Như vậy an toàn hơn, dù chậm hơn yêu cầu.

21. Bit-bang SPI

Hàm:

C

```
static uint8_t bitbang_transfer(uint8_t value)
```

Truyền từ bit 7 xuống bit 0:

C

```
for (int bit = 7; bit >= 0; --bit)
```

Đưa MOSI:

C

```
HAL_GPIO_WritePin(  
    AVR_MOSI_GPIO_Port,  
    AVR_MOSI_Pin,  
    value & (1u << bit)  
    ? GPIO_PIN_SET  
    : GPIO_PIN_RESET  
);
```

Chờ nửa chu kỳ:

C

```
AVRISP_PHY_DelayUs(g_half_period_us);
```

Đưa SCK lên:

C

```
HAL_GPIO_WritePin(  
    AVR_SCK_GPIO_Port,  
    AVR_SCK_Pin,  
    GPIO_PIN_SET  
);
```

Đọc MISO:

C

```
if (HAL_GPIO_ReadPin(...) == GPIO_PIN_SET)  
{
```



```
    rx |= 1u;  
}
```

Sau đó hạ SCK:

```
C  
  
HAL_GPIO_WritePin(  
    AVR_SCK_GPIO_Port,  
    AVR_SCK_Pin,  
    GPIO_PIN_RESET  
);
```

Đây là SPI mode 0:

```
CPOL = 0  
CPHA = 0
```

22. Delay microsecond bằng DWT

```
C  
  
void AVRISP_PHY_DelayUs(uint32_t us)
```

DWT là bộ đếm chu kỳ CPU trong Cortex-M3.

Số chu kỳ cần chờ:

```
C  
  
uint32_t cycles =  
    (SystemCoreClock / 1000000u) * us;
```

Nếu CPU 72 MHz:

```
1 µs = 72 cycle
```

Code đọc:

```
C  
  
DWT->CYCCNT
```

và chờ đủ số cycle.

Cách này chính xác hơn vòng lặp `for` thông thường.

23. Điều khiển RESET

C

```
void AVRISP_PHY_SetReset(bool asserted)
```

Có hỗ trợ cực tính:

C

```
g_reset_active_low
```

Với AVR thông thường:

RESET active-low

Do đó:

C

```
asserted = true
```

sẽ làm chân RESET xuống thấp.

Code:

C

```
if (g_reset_active_low)
{
    state = asserted
        ? GPIO_PIN_RESET
        : GPIO_PIN_SET;
}
```

Nên điều khiển qua transistor open-drain, không nhất thiết nối PB0 trực tiếp vào AVR.

24. Đo điện áp target

C

```
uint16_t AVRISP_PHY_ReadTargetMillivolts(void)
```

Khởi động ADC:

C

```
HAL_ADC_Start(&hadc1);
```

Chờ chuyển đổi:

C

```
HAL_ADC_PollForConversion(&hadc1, 10u);
```

Đọc giá trị:

C

```
uint32_t raw = HAL_ADC_GetValue(&hadc1);
```

Với bộ chia 47 kΩ/47 kΩ:

$$V_{ADC} = V_{TARGET} / 2$$

Công thức:

C

```
return (raw * 6600u + 2047u) / 4095u;
```

Giải thích:

ADC full scale = 3.3 V

Bộ chia ×2

VTARGET full scale = 6.6 V

Do đó:

$$V_{TARGET}(mV) = raw \times 6600 / 4095$$

25. Kiểm tra target có nguồn

C

```
uint8_t AVRISP_PHY_GetConnectionStatus(void)
```

Nếu điện áp nhỏ hơn 1,6 V:

```
C
if (mv < 1600u)
{
    return STATUS_TGT_NOT_DETECTED;
}
```

Nó báo target chưa được cấp nguồn.

Lưu ý: AVRISP mkII thật không cấp nguồn target. Nó chỉ đo điện áp target.

26. Vào chế độ ISP

Hàm:

```
C
static void cmd_enter(
    const uint8_t *p,
    uint16_t n)
```

AVR Studio truyền các tham số:

```
p[1] timeout
p[2] stabDelay
p[3] cmdexeDelay
p[4] synchLoops
p[5] byteDelay
p[6] pollValue
p[7] pollIndex
p[8..11] command Programming Enable
```

Command thường là:

```
AC 53 00 00
```

Trình tự vào ISP

Thả RESET:

```
C
```

```
AVRISP_PHY_SetReset(false);
```

Chờ ổn định:

C

```
AVRISP_PHY_DelayMs(stab);
```

Kéo RESET xuống:

C

```
AVRISP_PHY_SetReset(true);
```

Chờ trước khi gửi lệnh:

C

```
AVRISP_PHY_DelayMs(cmdexe);
```

Gửi bốn byte:

C

```
for (uint8_t i = 0; i < 4u; ++i)
{
    rx[i] = AVRISP_PHY_Transfer(p[8u+i]);
}
```

AVR thường phản hồi:

Byte thứ ba = 0x53

AVR Studio truyền `pollIndex` và `pollValue` để programmer biết cần kiểm tra byte nào.

Code đang dùng:

C

```
if (poll_index == 0u ||
    (poll_index <= 4u &&
     rx[poll_index-1u] == poll_value))
```

`pollIndex` trong giao thức là kiểu một-based:

```
1 → rx[0]
2 → rx[1]
3 → rx[2]
4 → rx[3]
```

27. Nếu vào ISP thất bại

Code thử lại:

C

```
AVRISP_PHY_SetReset(false);
AVRISP_PHY_DelayMs(2u);

AVRISP_PHY_SetReset(true);
AVRISP_PHY_DelayMs(20u);
```

Tối đa theo:

C

synchLoops

và timeout.

Nếu thành công:

C

```
in_progmode = 1u;
```

Nếu thất bại:

C

```
STATUS_CMD_FAILED
```

28. Load Address

AVR Studio gửi:

06 A3 A2 A1 A0

Code ghép địa chỉ:

```
C

current_address =
    ((uint32_t)p[1] << 24) |
    ((uint32_t)p[2] << 16) |
    ((uint32_t)p[3] << 8) |
    p[4];
```

Đối với Flash:

current_address là word address

Đối với EEPROM:

current_address là byte address

Bit 31:

```
C

address_is_extended =
    (current_address & 0x80000000u) ? 1u : 0u;
```

được dùng để báo chip có vùng địa chỉ Flash mở rộng.

29. Flash AVR dùng word address

Một word Flash AVR có hai byte:

Low byte
High byte

Ví dụ word address `0x0100` tương ứng hai byte Flash:

byte address `0x0200`
byte address `0x0201`

Trong code:

```
C

uint8_t hi = i & 1u;
```

- `i` chẵn: byte thấp.
- `i` lẻ: byte cao.

Sau byte cao:

```
C

if (hi)
{
    ++word;
}
```

30. Ghi Flash

Hàm:

```
C

static void cmd_program_flash(...)
```

Đọc số byte:

```
C

uint16_t count = be16(&p[1]);
```

Hàm `be16()` :

```
C

return ((uint16_t)p[0] << 8) | p[1];
```

Giao thức dùng big-endian.

Các trường:

```
C

uint8_t mode      = p[3];
uint8_t delay     = p[4];
uint8_t cmd_load  = p[5];
uint8_t cmd_write = p[6];
uint8_t cmd_read  = p[7];
uint8_t poll1     = p[8];
uint8_t poll2     = p[9];
```


Dữ liệu bắt đầu tại:

p[10]

Nạp dữ liệu vào page buffer AVR

Byte thấp thường dùng opcode:

0x40

Byte cao:

0x48

Code:

```
C
uint8_t tx[4] = {
    cmd_load | (hi ? 0x08u : 0u),
    word >> 8,
    word,
    p[10u+i]
};
```

Ví dụ:

40 00 10 AB

nạp 0xAB vào low byte của word address 0x0010 .

Ghi page thật vào Flash

Nếu bit 7 của mode được đặt:

```
C
if (mode & 0x80u)
```

thì gửi opcode write page:

```
C
```

```
uint8_t tx[4] = {
    cmd_write,
    start_word >> 8,
    start_word,
    0
};
```

Opcode thường là:

4C

Dữ liệu trong page buffer lúc này mới thực sự được ghi vào Flash.

31. Chờ Flash ghi xong

Có ba cách:

Timed delay

```
C

if (mode & 0x20u)
{
    AVRISP_PHY_DelayMs(delay);
}
```

Chờ cố định.

Value polling

```
C

else if (mode & 0x40u)
```

Đọc lại Flash cho đến khi bằng giá trị mong đợi.

Ready/busy polling

Nếu không thuộc hai cách trên:

```
C

wait_ready_busy()
```

gửi:

F0 00 00 00

và kiểm tra bit busy.

32. Đọc Flash

Hàm:

C

```
cmd_read_flash()
```

Với từng byte:

C

```
uint8_t hi = i & 1u;
```

Opcode:

C

```
read_opcode | (hi ? 0x08u : 0u)
```

Thông thường:

20 = đọc low byte

28 = đọc high byte

Kết quả nằm trong:

C

```
rx[3]
```

Response:

CMD_READ_FLASH_ISP

STATUS_CMD_OK

DATA...

STATUS_CMD_OK

Có hai status:

- một status đầu;

- một status cuối.

33. Flash lớn và Extended Address

Một số AVR như ATmega2560 có Flash lớn.

Hàm:

C

```
set_extended_address(word)
```

lấy phần địa chỉ cao:

C

```
uint8_t ext = word_addr >> 16;
```

Nếu thay đổi, gửi:

```
4D 00 EXT 00
```

Code nhớ giá trị trước:

C

```
static uint8_t last_ext_addr = 0xFFu;
```

để không gửi lại không cần thiết.

34. Ghi EEPROM

Hàm:

C

```
cmd_program_eeprom()
```

EEPROM dùng byte address, không phải word address.

Mỗi byte:

C

```
uint8_t tx[4] = {
    write_op,
    addr >> 8,
    addr,
    data
};
```

Opcode thường:

C0 addressH addressL data

Sau mỗi byte, code thực hiện:

- timed delay;
- value polling;
- hoặc ready/busy polling.

Sau đó tăng:

```
C
++addr;
```

35. Đọc EEPROM

```
C
uint8_t tx[4] = {
    read_op,
    addr >> 8,
    addr,
    0
};
```

Opcode đọc EEPROM thường:

A0

Kết quả:

```
C
response[2u+i] = rx[3];
```

36. Chip Erase

Hàm:

C

```
cmd_chip_erase()
```

AVR Studio cung cấp bốn byte opcode tại:

C

```
&p[3]
```

Thông thường:

```
AC 80 00 00
```

Code:

C

```
AVRISP_PHY_Transfer4(&p[3], rx);
```

Sau đó:

C

```
if (poll_method == 0u)
{
    delay
}
else
{
    ready/busy polling
}
```

37. Fuse và Lock bit

Ghi fuse/lock dùng:

C

```
cmd_program_misc()
```

Nó gửi thẳng bốn byte AVR Studio cung cấp:

C

```
AVRISP_PHY_Transfer4(&p[1], rx);
```

Ví dụ low fuse:

AC A0 00 VALUE

High fuse:

AC A8 00 VALUE

Lock:

AC E0 00 VALUE

Firmware không cần tự biết opcode cho từng chip vì AVR Studio truyền opcode đúng theo device database.

38. Đọc Fuse, Lock, Signature, OSCCAL

Hàm dùng chung:

C

```
cmd_read_misc()
```

Format:

```
COMMAND  
RET_INDEX  
SPI_BYTE_0  
SPI_BYTE_1  
SPI_BYTE_2  
SPI_BYTE_3
```

Sau khi truyền:

C

```
AVRISP_PHY_Transfer4(&p[2], rx);
```

Code chọn byte cần trả về:

```
C
response[2] =
    ret_index < 4u
    ? rx[ret_index]
    : 0u;
```

Ví dụ đọc signature:

```
30 00 INDEX 00
```

Kết quả thường nằm ở `rx[3]` .

39. CMD_SPI_MULTI

Đây là command cho phép AVR Studio gửi một chuỗi SPI tổng quát.

Các trường:

```
C
uint8_t tx_count_bytes = p[1];
uint8_t rx_count_bytes = p[2];
uint8_t rx_start       = p[3];
```

Ví dụ:

```
TX count = 4
RX count = 1
RX start = 3
```

Firmware truyền bốn byte rồi trả byte nhận được tại vị trí thứ ba.

Nếu cần nhận thêm byte sau khi hết dữ liệu TX, firmware gửi `0x00` :

```
C
uint8_t out =
    i < tx_count_bytes
    ? p[4u+i]
    : 0u;
```

40. Trình tự khởi tạo trong `main.c`

Bạn phải gọi CubeMX trước:

C

```
MX_GPIO_Init();
MX_ADC1_Init();
MX_SPI1_Init();
MX_USB_DEVICE_Init();
```

Sau đó:

C

```
AVRISP_MKII_Init();
```

Bên trong nó:

C

```
parameters[PARAM_RESET_POLARITY] = 1u;
parameters[PARAM_SCK_DURATION] = 6u;
AVRISP_PHY_Init();
```

Giá trị SCK duration 6 tương ứng gần:

125 kHz

Đây là tốc độ khởi động khá an toàn.

41. `usb_device.c` phải thay CDC

Code phải đăng ký:

C

```
USBD_RegisterClass(
    &hUsbDeviceFS,
    &USBD_AVRISP
);
```

Không đăng ký:

C

Nếu vẫn dùng CDC thì Windows chỉ thấy COM port và AVR Studio không nhận là AVRISP mkII.

42. VID/PID

Trong descriptor patch:

C

```
#define USBD_VID    0x03EB  
#define USBD_PID_FS 0x2104
```

Windows dùng hai giá trị này để tìm driver.

Chuỗi:

C

```
"AVRISP mkII"
```

chỉ là tên hiển thị. VID/PID mới là thông tin nhận dạng chính.

Lưu ý: dùng VID/PID của hãng khác chỉ phù hợp thử nghiệm cá nhân; sản phẩm thương mại phải có VID/PID hợp lệ riêng.

43. Điểm chưa hoàn toàn ổn trong code hiện tại

Có vài điểm cần hiểu trước khi sử dụng thực tế.

43.1 Message USB dài đúng bội số 64

Code kết thúc message khi:

C

```
len < 64
```

Nếu command dài đúng:

64, 128, 192, 256 byte

host cần gửi thêm ZLP để báo kết thúc.

Cần xác nhận driver AVR Studio thực sự gửi ZLP theo AVR069. Nếu không, nên xác định chiều dài command từ nội dung command thay vì chỉ dựa trên short packet.

43.2 TX response đúng bội số 64

Tương tự, nếu response dài đúng bội số 64, có thể cần gửi ZLP sau packet cuối để host biết message kết thúc.

Code hiện chưa có xử lý ZLP TX riêng.

43.3 TX queue đầy

```
C
if (tx_count >= TX_QUEUE_DEPTH)
{
    return;
}
```

Response sẽ bị bỏ nếu queue đầy. Nên sửa để:

- không lấy command mới khi TX queue đầy;
- hoặc tăng queue;
- hoặc dùng buffer command-response một đối một.

43.4 SPI tốc độ 8 MHz

STM32F103 chạy PCLK2 72 MHz không chia được đúng 8 MHz.

Các lựa chọn:

```
/8 = 9 MHz, quá cao
/16 = 4.5 MHz
```

Code chọn 4,5 MHz vì không muốn vượt yêu cầu.

Do đó mục 8 MHz trong bảng thực tế chạy khoảng 4,5 MHz.

43.5 Bit-bang rất chậm

Ở 50 Hz:

```
Một bit mất khoảng 20 ms
Một byte mất khoảng 160 ms
```

Giao dịch 4 byte có thể mất hơn nửa giây. Timeout AVR Studio cần phù hợp.

43.6 Đọc VTarget chỉ đúng với chia áp 47 kΩ/47 kΩ

Nếu bạn thay điện trở, công thức:

C

raw * 6600 / 4095

phải sửa.

43.7 RESET open-drain

Code đang cấu hình PB0 theo GPIO output bình thường. Với target nhiều mức điện áp, phần cứng tốt hơn là:

PB0 → transistor NPN/NMOS → RESET

PB0 chỉ kéo RESET xuống, còn điện trở target kéo lên VTarget.

44. Luồng đầy đủ khi AVR Studio đọc signature

1. PC gửi CMD_SIGN_ON
2. STM32 trả AVRISP_MK2
3. PC đọc version, VTarget và SCK
4. PC thiết lập SCK
5. PC gửi CMD_ENTER_PROGMODE_ISP
6. STM32 kéo RESET xuống
7. STM32 gửi AC 53 00 00
8. AVR phản hồi 0x53
9. STM32 trả STATUS_CMD_OK
10. PC gửi CMD_READ_SIGNATURE_ISP
11. STM32 gửi 30 00 00 00
12. STM32 nhận signature byte 0
13. Lặp với index 1 và 2
14. PC nhận ba byte signature
15. PC xác định model AVR

45. Luồng ghi chương trình

```
CMD_ENTER_PROGMODE_ISP
    ↓
CMD_CHIP_ERASE_ISP
    ↓
CMD_LOAD_ADDRESS
```

```

↓
CMD_PROGRAM_FLASH_ISP
↓
CMD_LOAD_ADDRESS
↓
CMD_READ_FLASH_ISP
↓
So sánh verify
↓
CMD_LEAVE_PROGMODE_ISP

```

Nếu verify sai, cần kiểm tra:

- SCK quá cao.
- MISO/MOSI đảo.
- RESET không giữ thấp.
- target chưa có nguồn.
- level shifter sai chiều.
- page address sai.
- polling kết thúc quá sớm.

46. Tóm tắt vai trò từng file

File	Chức năng
<code>avrisp_mkii.h</code>	API chính của firmware
<code>avr_isp_phy.h</code>	API phần cứng ISP
<code>usbd_avrisp.h</code>	Định nghĩa USB endpoint
<code>avrisp_mkii.c</code>	Xử lý command AVR069
<code>avr_isp_phy.c</code>	SPI, bit-bang, RESET, ADC
<code>usbd_avrisp.c</code>	USB Vendor Bulk class
<code>main_patch.txt</code>	Phần cần thêm vào <code>main.c</code>
<code>usb_device.c.patch.txt</code>	Đăng ký USB class AVRISP
<code>usbd_desc_patch.txt</code>	VID, PID và chuỗi USB

Bộ mã nguồn để đối chiếu trong khi đọc giải thích:

Download AVRISP mkII source

THEM PDI/TPI DI

Worked for 2m 36s

Đã thêm **PDI + TPI** vào firmware AVRISP mkII cho STM32F103:

Download STM32F103 AVRISP mkII ISP + PDI + TPI

Phần đã bổ sung

Giao thức XPROG của AVRISP mkII

Firmware nhận thêm:

C

```
#define CMD_XPROG          0x50
#define CMD_XPROG_SETMODE  0x51
```

Chọn giao diện:

C

```
#define XPROG_MODE_PDI     0x00
#define XPROG_MODE_TPI     0x01
```

Các lệnh con:

C

```
#define X_ENTER    0x01
#define X_LEAVE    0x02
#define X_ERASE    0x03
#define X_WRITE    0x04
#define X_READ     0x05
#define X_CRC      0x06
#define X_SETP     0x07
```

Nhóm lệnh TPI chuẩn gồm Enter, Leave, Erase, Write Memory, Read Memory và Set Parameter; dữ liệu nhiều byte dùng thứ tự MSB trước.  Micro... +1

File mới

```
Core/Inc/xprog.h
Core/Inc/pdi_phy.h
Core/Inc/tpi_phy.h
```

```
Core/Src/xprog.c
Core/Src/pdi_phy.c
Core/Src/tpi_phy.c
```

```
Docs/PDI_TPI_NOTES.md
```

PDI cho XMEGA

Chân mặc định:

STM32F103

XMEGA

PB10

PDI_CLK/RESET

PB11

PDI_DATA

GND

GND

VTarget sense

VCC target

PDI là giao diện hai dây: RESET được dùng làm `PDI_CLK`, còn `PDI_DATA` là đường dữ liệu hai chiều. Khung truyền dùng start bit, 8 data bit, even parity và stop bit; dữ liệu đổi ở cạnh xuống và được lấy mẫu ở cạnh lên của `PDI_CLK`.  Micro... +1

Đã có:

- Khung truyền PDI bằng GPIO.
- Even parity.
- Break character.
- Chuyển hướng `PDI_DATA` input/output.
- PDI NVM key:

C

```
FF 88 D8 CD 45 AB 89 12
```

- Kiểm tra `NVMEN`.
- Đọc vùng data space.
- Ghi vùng data space.
- Chip erase cơ bản.
- Poll NVM busy.
- Truy cập địa chỉ 32-bit.

PDI có thể truy cập Flash, EEPROM, fuse, lock bits và user signature thông qua NVM controller của XMEGA.  Micro...

TPI cho ATtiny

Chân mặc định:

STM32F103

ATtiny TPI

PB12

TPICLK

PB13

TPIDATA

PB14

Điều khiển switch RESET 12 V

GND

GND

TPI sử dụng ba tín hiệu:

RESET
TPICLK
TPIDATA

TPIDATA là đường half-duplex. Khung gồm một start bit, 8 bit dữ liệu LSB trước, even parity và hai stop bit.  Micro... +1

Đã có:

- TPI bit-bang.
- Even parity.
- Break character.
- TPIDATA open-drain/input.
- NVM enable key.
- Enter/Leave programming mode.
- Read memory.
- Write memory.
- Chip/Application/Configuration erase.
- Poll NVM busy.
- `TPI_PARAM_NVMCMD_ADDR` .
- `TPI_PARAM_NVMCSR_ADDR` .

Các parameter chuẩn tương ứng là:

C


```
#define TPI_PARAM_NVMCMD_ADDR 0x03
#define TPI_PARAM_NVCSR_ADDR 0x04
```

 Microc...

TPI target như ATtiny4/5/9/10 và ATtiny40 chỉ cho phép NVM programming ở 5 V; một số hệ thống TPI dùng 12 V trên RESET để kích hoạt programming, nên PB14 chỉ được dùng để điều khiển mạch switch bên ngoài.  Micro... +2

Cấu hình CubeMX bổ sung

PB10: GPIO Output
PB11: GPIO runtime input/open-drain
PB12: GPIO Output
PB13: GPIO runtime input/open-drain
PB14: GPIO Output, mặc định LOW

PB14 tuyệt đối không nối trực tiếp vào RESET chip:

```
graph TD
    PB14 --> Transistor[transistor/MOSFET điều khiển]
    Transistor --> Switch[switch 12 V]
    Switch --> RESET[RESET target]
```

Luồng command PDI/TPI

AVR Studio chọn PDI:

```
CMD_XPROG_SETMODE
XPROG_MODE_PDI
```

Hoặc TPI:

```
CMD_XPROG_SETMODE
XPROG_MODE_TPI
```

Sau đó:

```
CMD_XPROG
├── X_ENTER
├── X_ERASE
├── X_WRITE
├── X_READ
└── X_LEAVE
```

Giới hạn còn lại

Bản này đã thêm lớp giao thức và lớp vật lý thật, nhưng vẫn cần kiểm thử trên chip cụ thể trước khi ghi fuse:

- XPROG CRC hiện trả về chưa hỗ trợ.
- Một số lệnh NVM erase/write khác nhau giữa các họ XMEGA.
- Địa chỉ NVM controller phải đối chiếu datasheet từng dòng.
- TPI fuse/lock/config cần đúng memory address do AVR Studio cung cấp.
- Chưa kiểm thử thực tế với Atmel Studio và driver Jungo.
- Chưa thể khẳng định tương thích mọi XMEGA và mọi ATtiny TPI.

Nên thử theo thứ tự:

```
Read signature
Read Flash
Read fuse
Chip erase trên chip thử
Write Flash
Verify
Power cycle
```

Không nên thử ghi fuse hoặc lock bit đầu tiên.

ChatGPT can make mistakes. Check important info.